

CALLISTO · SECURITY REVIEW

Callisto Review.

A security review of Review.

CLIENT	PROJECT	AUDITORS	AUDIT WINDOW	FINAL REPORT
Callisto	Review	Guardian	October 13 – November 26, 2025	December 3, 2025

§ 00

Contents.

102 pages — overview, confidence ranking, scope, 64 findings, disclaimer, and about Guardian.

§ 01	Overview	P. 03
§ 02	Confidence Ranking	P. 04
§ 03	Scope & Classifications	P. 05
	Files in Scope	P. 06
§ 04	Findings	P. 07
	Main Review · 62 findings	P. 12
	Remediation Review · 2 findings	P. 98
§ 05	Disclaimer	P. 101
§ 06	About Guardian	P. 102

§ 01

Overview.

Guardian reviewed Review across 2 review rounds. The final report includes 4 Critical, 13 High, 18 Medium, 12 Low, 17 Informational findings.

AUDIT FIRM	REPOSITORY
Guardian	CallistoDA0/callisto-core
CLIENT	METHODOLOGY
Callisto	Smart contract · Manual · Main Review · Remediation Review
PROJECT	MAIN REVIEW COMMIT
Review	bd3ace38eb37
PREPARED BY	REMEDATION REVIEW COMMIT
Guardian	85aff9f0f030
AUDIT WINDOW	
October 13 – November 26, 2025	
FINAL REPORT	
December 3, 2025	

1.1 Confidence Ranking

1 / 5

Very Low Confidence

Guardian assigns a Confidence Ranking of 1 out of 5, combining finding severity, remediation outcome, and the reviewer’s assessment of latent risk. The full scale is on the next page.



1.2 Findings at a Glance

SEVERITY	TOTAL	PENDING	ACK	PARTIAL	RESOLVED
Critical	4	0	0	0	4
High	13	0	0	0	13
Medium	18	0	2	0	16
Low	12	0	1	0	11
Informational	17	0	5	0	12
All severities	64	0	8	0	56

§ 02

Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

CONFIDENCE RANKING	DEFINITION AND RECOMMENDATION	RISK PROFILE
5 Very High Confidence	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: code is highly secure at time of audit.	0 High/Critical findings and few Low/Medium severity findings.
4 High Confidence	Code is clean, well-structured, and adheres to best practices. Design patterns are sound and test coverage is strong. Recommendation: suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity findings.
3 Moderate Confidence	Code is functional but shows gaps in defensive programming or test coverage. Recommendation: remediate the findings and re-review the changes before deployment.	Some High/Critical findings, with various Medium severity findings.
2 Low Confidence	Recurring issues suggest systemic gaps rather than isolated mistakes. Recommendation: remediate, then commission a follow-up review at a frozen commit.	Over 6 High and Critical findings across the engagement.
1 Very Low Confidence ASSIGNED	The volume and severity of findings indicate the codebase is not ready for production. Recommendation: substantial rework, then a full re-review.	Over 12 High and Critical findings across the engagement.

§ 03

Scope & Classifications.

Guardian reviewed Review in `CallistoDA0/callisto-core`. The scope includes 4,245 source lines across 17 files.

3.1 Severity Matrix

	IMP · HIGH	IMP · MED	IMP · LOW
LIK · HIGH	Critical	High	Medium
LIK · MED	High	Medium	Low
LIK · LOW	Medium	Low	Low

IMPACT · HIGH
Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.

IMPACT · MEDIUM
Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.

IMPACT · LOW
Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.

LIKELIHOOD · HIGH
Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.

LIKELIHOOD · MEDIUM
Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.

LIKELIHOOD · LOW
Unlikely to ever occur in production.

Files in Scope.

FILE	NSLOC	TOTAL
src/policies/CallistoPSM.sol	217	380
src/policies/CallistoVault.sol	718	1,284
src/external/CDPRouter.sol	75	117
src/external/CallistoToken.sol	34	52
src/external/ConverterToWadDebt.sol	9	23
src/external/DebtTokenMigrator.sol	122	206
src/external/PSMStrategy.sol	551	1,036
src/external/VaultStrategy.sol	95	177
src/external/cdp/AdminContract.sol	278	480
src/external/cdp/BorrowerOperations.sol	268	497
src/external/cdp/CallistoChainlinkOracle.sol	88	183
src/external/cdp/ChainlinkCooler0LTVAdapter.sol	23	59
src/external/cdp/DebtToken.sol	100	191
src/external/cdp/LiquidationManager.sol	386	668
src/external/cdp/OHMDelegationManager.sol	251	489
src/external/cdp/StabilityPool.sol	384	860
src/external/cdp/VesselManager.sol	646	1,276

§ 04

Findings.

Grouped by round. Visible IDs reset by severity within each round, so a finding is named the same way in the report as it was in the channel it was written in.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
MAIN REVIEW					
C-01	Liquidation Debt Leak Via Index Scaling	Logical Error	CRITICAL	RESOLVED	p. 12
C-02	DoS Of Stability Pool	DoS	CRITICAL	RESOLVED	p. 14
C-03	Delegation Index Drift Blocks Liquidation	Unexpected Behavior	CRITICAL	RESOLVED	p. 15
C-04	Batch Liquidation Uses Stale Coll/Debt Indices	Logical Error	CRITICAL	RESOLVED	p. 17
H-01	Interest Miscount Bricks Vault Withdrawals	Logical Error	HIGH	RESOLVED	p. 19
H-02	Unsold Close Wraps collarDeficit	Logical Error	HIGH	RESOLVED	p. 20
H-03	Liquidator Cap Uses Wrong Units	Configuration	HIGH	RESOLVED	p. 22
H-04	Redemption Hints Break After Liquidations	Logical Error	HIGH	RESOLVED	p. 24
H-05	Migration Zeroes Principal Liquidity	Unexpected Behavior	HIGH	RESOLVED	p. 26
H-06	Empty PSMStrategy SP Balance	Unexpected Behavior	HIGH	RESOLVED	p. 28
H-07	Olympus Delegate Cap Blocks Depositors	Logical Error	HIGH	RESOLVED	p. 30
H-08	Donation Desyncs SP Deposit Ledger	Logical Error	HIGH	RESOLVED	p. 32
H-09	Zero-Gain Auction Blocks Liquidations	Logical Error	HIGH	RESOLVED	p. 33

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
H-10	Unsold Auctions Break When Deficit Cleared	DoS	HIGH	RESOLVED	p. 35
H-11	Router DoS When Vault Needs Debt Tokens	Unexpected Behavior	HIGH	RESOLVED	p. 37
H-12	Unsorted Liquidations	Logical Error	HIGH	RESOLVED	p. 39
H-13	Migration Zeroes Unsold Collateral	Unexpected Behavior	HIGH	RESOLVED	p. 41
M-01	Auto-Rescinds Freeze	Unexpected Behavior	MEDIUM	RESOLVED	p. 43
M-02	Wrong Debt Decimals Accounting	Math	MEDIUM	RESOLVED	p. 45
M-03	Arbitrary Swap Data	Logical Error	MEDIUM	RESOLVED	p. 46
M-04	Warm-up Griefing	DoS	MEDIUM	RESOLVED	p. 47
M-05	Unaccounted Interest	Logical Error	MEDIUM	RESOLVED	p. 48
M-06	Missing Liquidation Check In Execute	Access Control	MEDIUM	RESOLVED	p. 49
M-07	Emergency Burn Skews Excess Accounting	Logical Error	MEDIUM	RESOLVED	p. 50
M-08	Roles-dependent Contracts	Unexpected Behavior	MEDIUM	ACKNOWLEDGED	p. 51
M-09	repayDebtAndWithdrawColl Wrongly Reverts	Unexpected Behavior	MEDIUM	RESOLVED	p. 52
M-10	USDS Loss From Unclaimable Reimbursements	Unexpected Behavior	MEDIUM	RESOLVED	p. 53
M-11	Sub-share Withdrawals Steal Collateral	Rounding	MEDIUM	RESOLVED	p. 54
M-12	Unaccrued Interest In getRedemptionHints	Math	MEDIUM	RESOLVED	p. 56
M-13	USDS Profit Locked If Vault Is Liquidated	Logical Error	MEDIUM	RESOLVED	p. 57

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
M-14	Floor Check Dilutes Depositor Rewards	Validation	MEDIUM	RESOLVED	p. 58
M-15	Zero Heart Keeper Rewards	Logical Error	MEDIUM	RESOLVED	p. 60
M-16	Loss-rate Increment Can Block Liquidations	Math	MEDIUM	RESOLVED	p. 61
M-17	Cooler Principal Ledger Subtracts Interest	Logical Error	MEDIUM	ACKNOWLEDGED	p. 63
L-01	Deactivated Policies Keep Module Access	Unexpected Behavior	LOW	ACKNOWLEDGED	p. 65
L-02	Strategy Migration Misses Yield Vault Check	Validation	LOW	RESOLVED	p. 66
L-03	Timelock Delay Can Be Set To Zero	Warning	LOW	RESOLVED	p. 67
L-04	Collateral Withdraw Favors Users	Rounding	LOW	RESOLVED	p. 68
L-05	Setup Launches With Zero Borrow Fee	Configuration	LOW	RESOLVED	p. 69
L-06	Disabled Markets Still Mint Debt	Unexpected Behavior	LOW	RESOLVED	p. 70
L-07	Missing Debt Token Migration Check	Unexpected Behavior	LOW	RESOLVED	p. 71
L-08	Broken Functionality In borrowDebt	Logical Error	LOW	RESOLVED	p. 73
L-09	getGainsOf Miscomputes Pending Interest	Math	LOW	RESOLVED	p. 74
L-10	Finite Router Approvals Can Exhaust	Best Practices	LOW	RESOLVED	p. 76
L-11	deletedToBalance Reverts	Best Practices	LOW	RESOLVED	p. 77
I-01	Manager Sweep Can Drain Principal	Trust Assumptions	INFORMATIONAL	ACKNOWLEDGED	p. 78
I-02	Flashloan Guard Blocks Cross-calls Only	Warning	INFORMATIONAL	ACKNOWLEDGED	p. 79

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
I-03	Treasury Withdrawal Event Mislabels Actor	Warning	INFORMATIONAL	RESOLVED	p. 80
I-04	Heart Beat Bricks On Warmup Toggle	Unexpected Behavior	INFORMATIONAL	RESOLVED	p. 81
I-05	DoS If Yield Vault Rejects Deposits	Warning	INFORMATIONAL	RESOLVED	p. 83
I-06	Uninitialized Asset Updates Cause Event Noise	Events	INFORMATIONAL	RESOLVED	p. 85
I-07	Useless Check In addColl	Best Practices	INFORMATIONAL	RESOLVED	p. 86
I-08	Consider Using Or Removing The Unused Errors	Informational	INFORMATIONAL	RESOLVED	p. 87
I-09	Unused yieldVault Constructor Argument	Best Practices	INFORMATIONAL	RESOLVED	p. 88
I-10	setCollateralParameters Skips Activation Event	Warning	INFORMATIONAL	RESOLVED	p. 90
I-11	Unused Admin Slot	Best Practices	INFORMATIONAL	RESOLVED	p. 91
I-12	Dead Recovery Branch In WithdrawColl	Best Practices	INFORMATIONAL	RESOLVED	p. 92
I-13	Unused Instruction Struct In Kernel	Best Practices	INFORMATIONAL	ACKNOWLEDGED	p. 93
I-14	adminRole Slot Unused In ROLESv1	Best Practices	INFORMATIONAL	RESOLVED	p. 94
I-15	ensureValidRole Scans Padded Zeros	Best Practices	INFORMATIONAL	ACKNOWLEDGED	p. 95
I-16	End Price Unreachable At Auction Boundary	Logical Error	INFORMATIONAL	RESOLVED	p. 96
I-17	Missing Reserve	Logical Error	INFORMATIONAL	ACKNOWLEDGED	p. 97

REMEDIATION REVIEW

M-01	Migrate PSM Strategy Could Be DoSed	Validation	MEDIUM	RESOLVED	p. 98
------	-------------------------------------	------------	--------	----------	-------

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
L-01	Invalid Min Burn Amount Set	Configuration	LOW	RESOLVED	p. 100

CRITICAL **C-01**

RESOLVED

Liquidation Debt Leak Via Index Scaling

CATEGORY Logical Error
LOCATION VesselManager.sol
REVIEW Main Review

DESCRIPTION

`VesselManager.redistributeCollAndDebt` is responsible for pushing a liquidated borrower's remaining debt onto the surviving vessels. Today the function converts the incoming debt into "shares" with `SharesMathLib.toSharesDown(debt, state.debtIndexBelow0LTV)` and then bumps the debt index by `debtShares * RAY / totalShares`. Because that conversion divides by the current index, any index above 1.0 (which happens whenever interest has accrued) shrinks the debt before it is reassigned. A liquidation that should redistribute 3 debt tokens only raises the surviving positions by $3 \times (1 / \text{index})$, silently dropping the difference. The standalone Forge test at `test/external/cdp/VesselManager/VesselManagerRedistributionAccounting.t.sol` demonstrates the issue: after one year at 50% APR the index climbs to ~1.5, a liquidated borrower owes ~3 tokens, yet the surviving borrower only absorbs 2 and ~1 token vanishes. This accounting hole lets liquidated borrowers escape part of their debt and leaves the protocol under-collateralized even when the liquidation was solvent.

RECOMMENDATION

Increase `state.debtIndexBelow0LTV` using the raw debt amount rather than the share-scaled value, for example by adding `FixedPointMathLib.mulDiv(debt, SharesMathLib.RAY, state.debtSharesBelow0LTV)`, so every liquidation redistributes the full liability regardless of how large the index has grown:

```
- uint256 debtSharesBelow = SharesMathLib.toSharesDown(debt, state.debtIndexBelow0LTV);
uint256 debtSharesBelow0LTV = state.debtSharesBelow0LTV;
if (debtSharesBelow0LTV > 0) {
- state.debtIndexBelow0LTV += uint128((debtSharesBelow * SharesMathLib.RAY) /
debtSharesBelow0LTV);
+ uint256 increment = FixedPointMathLib.mulDiv(
+ debt,
+ SharesMathLib.RAY,
+ debtSharesBelow0LTV
+ );
```

C-01 Liquidation Debt Leak Via Index Scaling

CONTINUED

```
+ state.debtIndexBelow0LTV += uint128(increment);  
}
```

PROOF OF CONCEPT <https://gist.github.com/r0bert-ethack/459bf4a16fd2f1b8d33e99d63cb7ed39>

— RESOLUTION

Status: Resolved.

<https://github.com/CallistoDA0/callisto-core/pull/47>

CRITICAL C-02

RESOLVED

DoS Of Stability Pool

CATEGORY	DoS
LOCATION	https://github.com/GuardianOrg/callisto-core-team1-1759852427511/blob/main/src/external/cdp/StabilityPool.sol#L692
REVIEW	Main Review

DESCRIPTION

Users can deposit COLLAR into the Stability Pool to secure the protocol and earn a portion of interest paid by borrowers and collateral fees from liquidations. Upon deposits/withdrawals, the pool accrues the interest and distributes it to depositors using:

```
uint256 interest = DebtToken(debtToken).balanceOf(address(this)) - distributedInterest_ -
totalDeposits_;
if (interest != 0) updateSummary(_REWARD_TOKEN_POSITION, interest, totalDeposits, product_,
scale_, epoch_);
```

At a later stage, the asset gain is computed as follows:

```
uint256 numerator = amountToAdd * Constants.PERCENTAGE_PRECISION + err;
// slither-disable-start divide-before-multiply
rate = numerator / totalDeposits_;
```

However, it doesn't skip the division if `totalDeposits_` is zero.

This allows an attacker to borrow COLLAR, donate it to the Stability Pool when it is empty, then all future depositors will face a division by zero error when they try to deposit or withdraw, causing DoS.

RECOMMENDATION

Consider skipping the division if `totalDeposits_` is zero.

PROOF OF CONCEPT <https://gist.github.com/Ali-Shehab/968637ee18aa36c22199ac6d3b36f434>

RESOLUTION

Status: Resolved.

<https://github.com/CallistoDAO/callisto-core/pull/40>

CRITICAL **C-03**

RESOLVED

Delegation Index Drift Blocks Liquidation

CATEGORY Unexpected Behavior

LOCATION <https://github.com/GuardianOrg/callisto-core-team1-1759852427511/blob/bd3ace38eb37e1703a1f3bacd266e8f305b97e4d/src/external/cdp/OHMDelegationManager.sol#L445>

REVIEW Main Review

DESCRIPTION

When liquidating or reducing collateral, the system attempts to auto-rescind gOHM delegations by computing undelegated balance as:

```
newUndelegatedBalance = totalAccountGOhm - aState.delegatedGOhm;
```

totalAccountGOhm is recomputed on-the-fly from current collateral (cOHM → OHM via ERC-4626 round down, then OHM → gOHM via GOHM.balanceTo round down using the current sOHM/gOHM index). aState.delegatedGOhm is the stored absolute gOHM delegated at an earlier time. Index is assumed to be flat while it could actually increase.

OHMDelegationManager expresses balances in gOHM by converting cOHM → OHM and then OHM → gOHM via GOHM.balanceTo(OHM), which divides by sOHM.index(). In Olympus, sOHM.index() is monotonic non-decreasing: positive rebases increase _totalSupply, decrease _gonsPerFragment, and thus raise index(). Consequently, the same OHM later maps to fewer gOHM, so if a user previously delegated all at a smaller index, the freshly recomputed totalAccountGOhm during liquidation satisfies totalAccountGOhm ≤ aState.delegatedGOhm. The line newUndelegatedBalance = totalAccountGOhm - aState.delegatedGOhm; then underflows and reverts, blocking liquidation.

Impact: Any borrower who has ever delegated 100% of their cOHM can become non-liquidatable after any positive rebase, creating a DoS on liquidations and potential bad-debt accumulation. This is intentional-attack feasible: delegate-all → wait one rebase/force one rebase → liquidation attempts revert.

RECOMMENDATION

Consider replacing this code:

```
uint256 newUndelegatedBalance = totalAccountGOhm - aState.delegatedGOhm;
```

with:

```
uint256 newUndelegatedBalance = totalAccountGOhm > aState.delegatedGOhm ? (totalAccountGOhm - aState.delegatedGOhm) : 0;
```

C-03 Delegation Index Drift Blocks Liquidation

CONTINUED

In every place of the code where current `totalAccountGOhm` is compared to the old `aState.delegatedGOhm`.

— RESOLUTION

Status: Resolved.

<https://github.com/CallistoDA0/callisto-core/pull/59>

CRITICAL C-04

RESOLVED

Batch Liquidation Uses Stale Coll/Debt Indices

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/callisto-core-team1-1759852427511/blob/main/src/external/cdp/LiquidationManager.sol#L188-L190
REVIEW	Main Review

DESCRIPTION

When a liquidation occurs, part of the debt and collateral is redistributed to the remaining vessels. This increases both the debt and collateral indices so vessels account for the redistributed amounts. During liquidation, the protocol calculates collateral compensation for the liquidator and the protocol based on the entire vessel collateral, and the remainder is distributed either to the Stability Pool (SP) or to other vessels:

```
debtToOffset = FixedPointMathLib.min(debt, debtTokenInStabPool);
collToSendToSP = (coll * debtToOffset) / debt;
debtToRedistribute = debt - debtToOffset;
collToRedistribute = coll - collToSendToSP;
```

The liquidation also closes the liquidated vessel. The protocol uses the collateral and debt indices to compute the correct amounts when closing the vessel, handled in `_decreaseVesselAmounts`.

This works correctly for single-vessel liquidations. However, the protocol also allows liquidating multiple vessels in a single transaction (up to a maximum). In multi-vessel liquidations, the protocol accumulates the total collateral and debt to redistribute and only calls `redistributeCollAndDebt` once, after all vessels are processed, in `_finalizeTotalLiquidation`:

```
if (total.debtToRedistribute > 0 || total.collToRedistribute > 0) {
  p.vm.redistributeCollAndDebt(asset, total.collToRedistribute, total.debtToRedistribute);
}
```

This is incorrect because the collateral and debt indices are updated only once at the end. While closing each vessel in the batch, the protocol uses stale indices that do not reflect redistribution from earlier vessels in the same batch.

C-04 Batch Liquidation Uses Stale Coll/Debt Indices

CONTINUED

This leads to incorrect calculations of the ICR when liquidating multiple vessels, and can result in under-collateralized vessels not being liquidated due to using stale indices.

Consequently, batch liquidations can yield different outcomes than sequential one-by-one processing; a position that would be liquidatable when processed individually may become non-liquidatable in a batch due to stale indices applied within the same transaction.

— RECOMMENDATION

Consider updating the collateral/debt indices and the aggregate entire debt/collateral after each vessel is liquidated within the batch, rather than only once at the end. This ensures every vessel in the batch is evaluated and settled using the most up-to-date indices.

PROOF OF CONCEPT <https://gist.github.com/Ali-Shehab/548d8307429d9bd91431bbb41639bf8b>

— RESOLUTION

Status: Resolved.

<https://github.com/CallistoDA0/callisto-core/pull/51>

HIGH **H-01**

RESOLVED

Interest Miscalculation Bricks Vault Withdrawals

CATEGORY Logical Error
LOCATION CallistoVault.sol
REVIEW Main Review

DESCRIPTION

`CallistoVault._withdrawStrategyOrCallerFunds` decides whether the strategy can cover the Cooler repayment by comparing `debtToRepay` against `strategyBalance`, but it computes that balance as `strategy.totalAssetsAvailable() + interest`. `totalAssetsAvailable()` already returns the strategy's actual withdrawable principal (capped by `maxWithdraw`). The extra interest term is merely the amount owed to Cooler, not a token balance stored in the strategy. When interest accrues faster than realised profit, the sum exceeds what the yield vault can return, so the function enters the "strategy covers everything" branch and calls `strategy.divest(debtToRepay, ...)`. That call propagates to `yieldVault.withdraw(debtToRepay, ...)`, which reverts because the vault only holds the principal. The public withdraw path therefore reverts and traps user funds even though they are entitled to them.

RECOMMENDATION

Base the availability check solely on tokens the strategy can actually withdraw (e.g., `strategy.totalAssetsAvailable()` and, if needed, realised profit). If that amount is smaller than the repayment requirement, withdraw only the available balance and collect the shortfall from the caller instead of mixing in the "interest" figure.

PROOF OF CONCEPT <https://gist.github.com/r0bert-ethack/79274bfab92f3d4fdab1b93f74fe1618>

RESOLUTION

Status: Resolved.

<https://github.com/CallistoDAO/callisto-core/pull/68>

HIGH **H-02**

RESOLVED

Unsold Close Wraps collarDeficit

CATEGORY Logical Error
LOCATION PSMStrategy.sol
REVIEW Main Review

DESCRIPTION

`closeUnsoldAuctions` blindly computes a deficit for every expired auction, even those created via `addAuctionForUnsold` where `target` is set to zero. When such a zero-target auction sells a portion of its collateral, `raised` becomes positive and the following code wraps:

```
remainingCollateral = rAuction.capacity;  
deficit = _calcRemainingCOLLAR(rAuction.target, rAuction.raised);  
unsoldCollateral[collateral] += remainingCollateral;  
delete rAuction.capacity;  
totalDeficit += deficit;
```

The helper just performs an unchecked subtraction:

```
function _calcRemainingCOLLAR(uint256 target, uint256 raised) private pure returns (uint256)  
{  
    return FPMath.rawSub(target, raised);  
}
```

Because `FPMath.rawSub(0, raised)` underflows to `2256 - raised`, the wrap propagates into `totalDeficit` and then `collarDeficit`, permanently corrupting the deficit ledger. The fork PoC in `PSMStrategyCloseUnsoldDeficit.t.sol` demonstrates the exact wraparound, after which all future unsold auctions mis-account proceeds.

RECOMMENDATION

Skip this deficit path for zero-target auctions (e.g., guard the subtraction with `if (rAuction.target != 0)`) or replace it with a checked subtraction/revert, ensuring unsold auctions can never feed negative deltas into `collarDeficit`.

PROOF OF CONCEPT <https://gist.github.com/r0bert-ethack/952632dd148829a5118febe3f684565b>

H-02 Unsold Close Wraps collarDeficitCONTINUED

— RESOLUTION

Status: Resolved.

<https://github.com/CallistoDA0/callisto-core/pull/73>

HIGH **H-03**

RESOLVED

Liquidator Cap Uses Wrong Units

CATEGORY Configuration
LOCATION LiquidationManager.sol
REVIEW Main Review

DESCRIPTION

`LIQUIDATOR_COLL_FEE_CAP_DEFAULT` and its min constant are stored as plain integers `(uint128(1000)/uint128(10))` even though the comment states they are “denominated in debt tokens”:

```
/// @dev Liquidator fee cap values are denominated in debt tokens; collateral limits are enforced using oracle
```

```
/// prices.
```

```
uint128 internal constant LIQUIDATOR_COLL_FEE_CAP_DEFAULT = uint128(1000); // 1000
```

```
uint128 internal constant LIQUIDATOR_COLL_FEE_CAP_MAX = type(uint128).max;
```

```
uint128 internal constant LIQUIDATOR_COLL_FEE_CAP_MIN = uint128(10); // 10
```

When a collateral is registered, `LiquidationManager.addCollateralType` seeds `collateralParams[asset].liquidatorCollFeeCap` with that raw number:

```
/**  
 * @notice Adds a new collateral type  
 * @param asset The collateral asset address to add  
 */  
function addCollateralType(address asset) external {  
    _checkAdminContract();  
    CollateralParams storage state = collateralParams[asset];  
    state.liquidatorCollFee = Constants.LIQUIDATOR_COLL_FEE_DEFAULT;  
    _setParams(  
        asset,  
        CollateralParams({  
            liquidatorCollFee: Constants.LIQUIDATOR_COLL_FEE_DEFAULT,  
            liquidatorCollFeeCap: Constants.LIQUIDATOR_COLL_FEE_CAP_DEFAULT,  
            protocolCollFee: Constants.PROTOCOL_COLL_FEE_DEFAULT  
        })  
    );  
}
```

H-03 Liquidator Cap Uses Wrong Units

CONTINUED

During liquidation `_getCollComps` converts the cap back to collateral via

```
MarketMath.debtToCollDown(params.liquidatorCollFeeCap, price);:
```

```
function _getCollComps(address asset, uint256 price, uint256 entireColl)
internal
view
returns (uint256 liquidatorCollComp, uint256 protocolCollComp)
{
    CollateralParams storage params = collateralParams[asset];
    liquidatorCollComp = MarketMath.mulPercentDown(entireColl, params.liquidatorCollFee);
    uint256 liquidatorCollCompCap = MarketMath.debtToCollDown(params.liquidatorCollFeeCap,
price);
    if (liquidatorCollComp > liquidatorCollCompCap) {
        protocolCollComp = liquidatorCollComp - liquidatorCollCompCap;
        liquidatorCollComp = liquidatorCollCompCap;
    }
    protocolCollComp += MarketMath.mulPercentDown(entireColl,
collateralParams[asset].protocolCollFee);
}
```

`debtToCollDown` multiplies by `Constants.ORACLE_PRICE_SCALE` (1e18) before dividing by the oracle price, so it expects the debt input to use 18-decimal precision, just like COLLAR's ERC20 accounting. Because the default cap is 1000 wei instead of 1000e18, the conversion yields about 1e-6 collateral tokens at a \$10 price, while the configured 3% liquidator fee on a typical vessel would be hundreds of tokens. That cap fires on every liquidation, shrinking the liquidator reward to dust and rerouting the remainder to the protocol, which removes the economic incentive to liquidate and risks leaving undercollateralised positions open.

— RECOMMENDATION

Store the cap constants in the same 18-decimal units the debt token uses (e.g. `uint128(1000 ether)` for the default and `uint128(10 ether)` for the minimum), and ensure any governance updates or migration code applies the same scaling so liquidator payouts reflect the intended maximum.

— RESOLUTION

Status: Resolved.

<https://github.com/CallistoDA0/callisto-core/pull/52>

HIGH **H-04**

RESOLVED

Redemption Hints Break After Liquidations

CATEGORY Logical Error
LOCATION VesselManager.sol
REVIEW Main Review

DESCRIPTION

`VesselManager.getRedemptionHints` subtracts the collateral lot in asset units and then feeds that value into `MarketMath.computeNominalCR` alongside share-denominated debt:

```
uint256 newColl = col - collLot;  
...  
partialRedemptionHintNewNICR = MarketMath.computeNominalCR(newColl, syntheticDebtShares);
```

After `redistributeCollAndDebt` runs during a liquidation, `collIndex` increases, so `newColl` must be converted back to shares. Without that conversion the computed hint drifts from the real post-redemption NICR and the check in `redeemCollateral` reverts with `VM_UnableToRedeemAnyAmount`. The PoC attach shows a redemption succeeding before liquidation, then reverting after a realistic `LiquidationManager.batchLiquidateVessels` call. Therefore, post-liquidation, hint-guided redemptions revert indefinitely, blocking core deleveraging flows.

RECOMMENDATION

Convert the simulated collateral into shares before computing the partial NICR so both operands use the same basis:

```
uint256 newCollShares = SharesMathLib.toSharesDown(newColl, state.collIndex);  
uint256 nextNICR = MarketMath.computeNominalCR(newCollShares, syntheticDebtShares);
```

Keeping the units aligned preserves redemption hints even after `collIndex` grows.

PROOF OF CONCEPT <https://gist.github.com/r0bert-ethack/16afc83da540b78fb1b2d61d09fc30eb>

RESOLUTION

Status: Resolved.

H-04 Redemption Hints Break After LiquidationsCONTINUED

<https://github.com/CallistoDA0/callisto-core/pull/56>

HIGH **H-05**

RESOLVED

Migration Zeroes Principal Liquidity

CATEGORY Unexpected Behavior
LOCATION VaultStrategy.sol
REVIEW Main Review

DESCRIPTION

The `VaultStrategy.migrate(address newStrategy)` function deletes `principalAssets`, moves the ERC4626 shares to the new contract and repoints the vault/PSM:

```
function migrate(address newStrategy) external nonzeroAddress(newStrategy) {  
    _requireRole(Roles.MIGRATOR);  
    emit PrincipalAssetsChange(-principalAssets);  
    delete principalAssets;  
    IERC4626 yieldVault_ = yieldVault;  
    uint256 balance = yieldVault_.balanceOf(address(this));  
    if (balance != 0) yieldVault_.safeTransfer(newStrategy, balance);  
    VAULT.migrateToNewStrategy(newStrategy);  
    PSM.migrateLiquidityProvider(newStrategy);  
    emit Migrated(newStrategy);  
}
```

The replacement strategy now owns the debt-token shares but reports `principalAssets == 0`. PSM redemptions query that value:

H-05 Migration Zeroes Principal Liquidity

CONTINUED

```
function swapIn(address to, uint256 assets) external override returns (uint256 collarIn) {
    ...
    uint256 available = availableLiquidity();
    require(assets <= available, PSM_InsufficientLiquidity(assets, available));
    ...
}

function availableLiquidity() public view override returns (uint256) {
    return VaultStrategy(liquidityProvider).totalAssetsAvailable();
}

function totalAssetsAvailable() public view override returns (uint256) {
    if (principalAssets < 0) return 0;
    return Math.min(uint256(principalAssets), yieldVault.maxWithdraw(address(this)));
}
```

With `principalAssets` zeroed, `availableLiquidity()` returns zero; every `swapIn` reverts once the migration executes. Users can no longer redeem COLLAR or repay CDP debt, Stability Pool top-ups stall and the peg backstop vanishes. Vault positions stay collateralized, so no immediate liquidations occur, but redemption flow is permanently frozen, risking a peg drift.

This concrete behaviour is bypassed in your integration/fork tests by the use of the `VaultStrategyTester` mock contract which implements a mock `addLiquidity` function that calls `_updatePrincipalAssets(int256(assets));`.

— RECOMMENDATION

Add a migration-aware principal sync. Either enhance `VaultStrategy.migrate` to accept the carried principal and set it on the new strategy immediately, or introduce a single-use governance initializer (e.g., `finalizeMigration(int256 principalAssets)`) callable straight after the pointer swap. Ensure the migration runbook invokes this step so `principalAssets` and `availableLiquidity` stay accurate and `swapIn` keeps working.

PROOF OF CONCEPT <https://gist.github.com/r0bert-ethack/9a65862bccd148472df3e9ba5b66732f>

— RESOLUTION

Status: Resolved.

<https://github.com/CallistoDA0/callisto-core/pull/92>

HIGH **H-06**

RESOLVED

Empty PSMStrategy SP Balance

CATEGORY	Unexpected Behavior
LOCATION	https://github.com/GuardianOrg/callisto-core-team1-1759852427511/blob/main/src/policies/CallistoPSM.sol
REVIEW	Main Review

— DESCRIPTION

The PSM allows anyone to burn "excess" COLLAR via a public function that, after consuming PSM-held pending tokens, withdraws up to the entire SP deposit of the PSMStrategy and burns it. Because `swapOut()` (USDS→COLLAR) immediately credits the minted amount as "excess", an attacker can perform:

`swapOut(large)` → increases `excessCOLLAR` by the minted amount,

`burnExcessCOLLAR()` (public) → burns pending and withdraws & burns up to all `withdrawableCOLLAR()` from the Stability Pool (SP).

This can be done right before a liquidation, reducing the PSMStrategy's SP deposit to ~0, so the strategy receives no liquidation proceeds for that round. Since auctions are created only when the PSMStrategy itself receives liquidation collateral, no new SP-sourced auctions are created for that window, preventing the recycle loop (collateral → auction → COLLAR paid → re-deposit → surplus to Treasury).

Attack Path :

Just before a liquidation (frontrun):

Call `swapOut(attacker, USDS_amount_large)` → mints `collarOut` and `excessCOLLAR += collarOut`.

Call `burnExcessCOLLAR()`:

burns `collarPendingBurning`, then

reads `toBurn = min(strategy.withdrawableCOLLAR(), excess)`,

`strategy.withdrawCOLLAR(toBurn)`; PSM burns that amount.

Liquidation executes; PSMStrategy SP balance ~0 → receives 0 collateral, so no `addAuction(...)` call is made for the strategy.

Result: Protocol misses proceeds & auctions for that round; attacker/others in SP get a larger pro-rata share.

H-06 Empty PSMStrategy SP Balance

CONTINUED

RECOMMENDATION

Restrict public burn to PSM-held pending only; gate strategy withdrawals behind keeper with SP floor/caps/cooldown;

RESOLUTION

Status: Resolved.

<https://github.com/CallistoDAO/callisto-core/pull/42>

HIGH **H-07**

RESOLVED

Olympus Delegate Cap Blocks Depositors

CATEGORY Logical Error
LOCATION CallistoVault.sol
REVIEW Main Review

DESCRIPTION

Callisto routes every gOHM delegation through `OHMDelegationManager.applyDelegations`, which forwards the batch to Olympus Cooler on behalf of the vault itself:

```
return OLYMPUS_COOLER.applyDelegations({
  delegationRequests: requests,
  onBehalfOf: address(this) // CallistoVault
});
```

Olympus' DLGTE module enforces a hard per-account limit (`DEFAULT_MAX_DELEGATE_ADDRESSES`, default 10) whenever it sees a new delegate:

```
(bool alreadyExisted, uint256 existingAmount) = acctDelegatedAmounts.tryGet(delegate);
if (!alreadyExisted && acctDelegatedAmounts.length() >= maxDelegates) {
  revert DLGTE_TooManyDelegates();
}
// https://vscode.blockscan.com/ethereum/0xD3204Ae00d6599Ba6e182c6D640A79d76CdAad74:
_addDelegation
```

Because every delegation Callisto submits uses the same `onBehalfOf = address(CallistoVault)`, that global cap is shared across all depositors. Once ten unique delegates have been added, the eleventh request reverts, even if the depositor is using their own first delegate and Callisto's local per-user cap (default 3) has plenty of room. Olympus enforces this before Callisto's manager can react, so subsequent `applyDelegations` calls fail and every later user is prevented from assigning their gOHM voting power.

Impact: the moment the shared pool reaches 10 delegates, delegation is broken for everyone who comes after.

RECOMMENDATION

Have governance lift the Olympus cap for `address(CallistoVault)`, either by calling `IDLGTEv1.setMaxDelegateAddresses(account, newMax)` on the DLGTE policy or `IMonoCooler.setMaxDelegateAddresses(address(this), newMax)` via the Cooler, so the shared pool can

H-07 Olympus Delegate Cap Blocks Depositors

CONTINUED

accommodate hundreds or thousands of delegates. Align the value with expected user counts and consider adding an initialization check that asserts the on-chain cap meets Callisto's per-user limit × anticipated users to catch misconfiguration early.

— RESOLUTION

Status: Resolved.

HIGH **H-08**

RESOLVED

Donation Desyncs SP Deposit Ledger

CATEGORY Logical Error
LOCATION PSMStrategy.sol
REVIEW Main Review

DESCRIPTION

`donateCOLLARToReduceDeficit` accepts COLLAR, forwards it into the `StabilityPool` and decrements both `collarDeficit` and `collarPendingReplenishment`, but it never raises the internal ledger that tracks the live SP principal. The function ends with the deposit call:

```
STABILITY_POOL.deposit(collarAmount, address(this));
```

while leaving the pre-existing `collarInSP` variable unchanged at its old value. During the next liquidation the `StabilityPool` invokes `addAuction`, providing `updatedSPDeposit = calcCompoundedDeposit(address(this))`, which now reflects the donation. The strategy immediately computes `depositPendingReplenishment = collarInSP - updatedSPDeposit`, so the stale ledger forces a negative subtraction whenever the liquidation consumes less than the donated amount and the transaction reverts. If the subtraction succeeds, the deficit ledger remains understated by the unspent portion, leaving later replenishment logic working off a stale balance. In both cases the auction accounting loses sync with the `StabilityPool` and deficit recovery for that liquidation cycle stalls.

RECOMMENDATION

Whenever external COLLAR is donated, bump `collarInSP` by the same `collarAmount` (without touching `collarFromPSM`) so the internal accounting mirrors the actual `StabilityPool` balance before any later redistributions.

RESOLUTION

Status: Resolved.

<https://github.com/CallistoDA0/callisto-core/pull/37>

HIGH **H-09**

RESOLVED

Zero-Gain Auction Blocks Liquidations

CATEGORY Logical Error
LOCATION StabilityPool.sol; PSMStrategy.sol
REVIEW Main Review

DESCRIPTION

During a liquidation the `StabilityPool` calls `_addAuctionInPSM`, which immediately forwards the collateral gain it just claimed to `PSMStrategy.addAuction`. The helper relies on `_calcRateAndError` to pro-rate gains across deposits. For minuscule offsets, that routine floors the collateral gain rate to zero even though the debt-loss rate remains ≥ 1 , so the PSM deposit shrinks (`depositPendingReplenishment > 0`) while the claimed gain is exactly zero:

```
// src/external/cdp/StabilityPool.sol:436-437
IPSMStrategy(psmStrategyAddr).addAuction(
    collateral,
    gains[collateralPosition], // may be 0
    collateralPrice,
    updatedDeposit
);
```

Inside `PSMStrategy.addAuction`, the code computes the auction's end price as:

```
// src/external/PSMStrategy.sol:505-507
uint256 endPrice = depositPendingReplenishment.mulDiv(
    Constants.ORACLE_PRICE_SCALE,
    collateralAmount,
    Math.Rounding.Ceil
);
```

When `collateralAmount == 0`, `mulDiv` divides by zero and reverts. Because `addAuction` is invoked within `StabilityPool.offset`, the entire liquidation reverts, preventing the system from processing liquidations until a larger gain materialises. Impact: small liquidations or rounding dust can DoS the liquidation path, leaving bad debt in the system.

RECOMMENDATION

H-09 Zero-Gain Auction Blocks Liquidations

CONTINUED

Handle the zero-gain case before invoking `addAuction`. For example, in `_addAuctionInPSM`, skip auction creation when `gains[collateralPosition] == 0` and accumulate the deficit in `collarDeficit / unsoldCollateral`. Alternatively, guard inside `PSMStrategy.addAuction` by requiring a nonzero `collateralAmount` (`require(collateralAmount != 0, PSMStrategy_ZeroCollateral())`) and moving the deficit bookkeeping there so the liquidation never reverts.

— RESOLUTION

Status: Resolved.

<https://github.com/CallistoDAO/callisto-core/pull/74>

HIGH **H-10**

RESOLVED

Unsold Auctions Break When Deficit Cleared

CATEGORY DoS
LOCATION PSMStrategy.sol
REVIEW Main Review

DESCRIPTION

When `_purchase` handles an “unsold” auction (`target = 0`) it reduces `collarDeficit` by the buyer’s payment and only deposits the portion needed back into the `StabilityPool`:

```
if (deficit >= collarPayment) {  
    collarDeficit = deficit - collarPayment;  
    collarPaymentWoProfit = collarPayment;  
} else {  
    uint256 collarProfit = collarPayment - deficit;  
    collarPaymentWoProfit = deficit; // zero once the deficit is cleared  
    delete collarDeficit;  
    treasuryProfit[COLLAR] += collarProfit;  
}
```

Even when the deficit is already zero, the code falls through with `collarPaymentWoProfit == 0` and `_purchase` still calls `_settleTransfersAndRedepositToSP(collarPayment, collarPaymentWoProfit, ...)`. That helper invokes `STABILITY_P00L.deposit(collarPaymentWoProfit, address(this))`. The `StabilityPool` rejects zero deposits, so the transaction reverts. From that moment every purchase against that auction reverts and the remaining collateral becomes stuck in `unsoldCollateral`.

RECOMMENDATION

Guard the settlement path when `collarPaymentWoProfit` is zero. Skip the SP deposit and only transfer collateral or prevent further purchases once the deficit is cleared by forcing admins to close the auction and handle the remainder manually. The key is to avoid calling `STABILITY_P00L.deposit(0, ...)` after the deficit has been fully repaid.

PROOF OF CONCEPT <https://gist.github.com/r0bert-ethack/acb2b3d9d2945d88602948c5f9b854df>

H-10 Unsold Auctions Break When Deficit Cleared

CONTINUED

— RESOLUTION

Status: Resolved.

<https://github.com/CallistoDA0/callisto-core/pull/74>

HIGH **H-11**

RESOLVED

Router DoS When Vault Needs Debt Tokens

CATEGORY Unexpected Behavior
LOCATION CDPRouter.sol
REVIEW Main Review

DESCRIPTION

When `CallistoVault.redeem` can't cover a withdrawal out of the buffered OHM, it enters the Cooler unwind path inside `_withdraw` and explicitly sources debt tokens before unstaking collateral:

```
uint256 ohmBalance = pendingOHMDeposits;
if (assets <= ohmBalance) {
  ... // direct payout
} else {
  emit PendingOHMDepositsChanged(-int256(ohmBalance), 0);
  delete pendingOHMDeposits;
  ...
  (uint128 wadDebt, uint256 debtToRepay) = _calcDebtToRepay(gOHMAmountToWithdraw);
  ...
  _withdrawStrategyOrCallerFunds(
    debtToRepay, debtToken_, strategy_.totalAssetsAvailable() + interest, strategy_
  );
}
```

If the strategy cannot cover `debtToRepay`, `_withdrawStrategyOrCallerFunds` asks the caller to front the difference:

```
if (debtToRepay <= strategyBalance) {
  if (debtToRepay != 0) strategy_.divest(debtToRepay, address(this));
  return;
}
uint256 callerContribution = FixedPointMathLib.rawSub(debtToRepay, strategyBalance);
debtToken_.safeTransferFrom(msg.sender, address(this), callerContribution);
```

The router invokes `redeem` with `msg.sender == address(this)`:

H-11 Router DoS When Vault Needs Debt Tokens

CONTINUED

```
if (collToWithdraw != 0) {  
  VAULT.redeem(collToWithdraw, msg.sender, address(this));  
}
```

During a user exit, once profits have been swept or liquidity is low, the vault reaches the `safeTransferFrom` call above. Because the router never escrows or approves debt tokens to the vault, the transfer reverts with an `Usds/insufficient-balance` error. Any router withdrawal that needs to repay Cooler debt is therefore blocked.

— RECOMMENDATION

Adjust the router so it pre-pulls debt tokens from the user and approves/transfers the exact shortfall to the vault before calling `redeem`, or modify the vault to charge the end user directly instead of `msg.sender`. This guarantees `_withdrawStrategyOrCallerFunds` can obtain the needed debt tokens even when the strategy balance is exhausted.

PROOF OF CONCEPT <https://gist.github.com/r0bert-ethack/a436cb1b1a1e36bc658d743f065049db>

— RESOLUTION

Status: Resolved.

<https://github.com/CallistoDAO/callisto-core/pull/84>

HIGH **H-12**

RESOLVED

Unsorted Liquidations

CATEGORY	Logical Error
LOCATION	LiquidationManager.sol
REVIEW	Main Review

DESCRIPTION

When in Recovery Mode, the protocol performs a special liquidation that checks a vessel's ICR and applies different strategies accordingly. The case of interest is when `(ICR >= p.MCR) && (ICR < TCR) && (entireVesselDebt <= p.debtInSP)`. In this case, the vessel's entire debt is offset against the debt in SP, and the collateral is capped at MCR. The remaining collateral becomes claimable by the vessel owner as collateral surplus. This occurs without any collateral/debt redistribution.

Separately, the protocol allows both sorted and arbitrary liquidations in both normal and Recovery Modes. An arbitrary liquidation processes vessels in an unspecified order.

Consider multiple users who satisfy the above condition, with `ICR_1 > ICR_2`, both above MCR and below TCR. To favor the protocol, liquidation should target the lowest ICR first, as it increases TCR the most. However, if an arbitrary liquidation hits the higher-ICR vessel first, it will offset all its debt against the SP and leave collateral as surplus. If the SP balance was exactly equal to the liquidated debt, the following check will subsequently evaluate as true and skip the lower-ICR vessel:

```
// Skip this vessel if ICR is greater than MCR and the Stability Pool is empty
if (ICR >= p.MCR && p.debtInSP == 0) {
    continue;
}
```

This keeps the protocol stuck in Recovery Mode. Had a sorted liquidation processed the lowest-ICR vessel first, it would have exited Recovery Mode.

RECOMMENDATION

Consider blocking arbitrary liquidations while in Recovery Mode, allowing only sorted liquidations.

PROOF OF CONCEPT <https://gist.github.com/Ali-Shehab/924a3a945bb717ad5b3317c05330eef4>

H-12 Unsorted Liquidations

CONTINUED

— RESOLUTION

Status: Resolved.

<https://github.com/CallistoDA0/callisto-core/pull/57>

HIGH **H-13**

RESOLVED

Migration Zeroes Unsold Collateral

CATEGORY Unexpected Behavior
LOCATION PSMStrategy.sol
REVIEW Main Review

DESCRIPTION

The `PSMStrategy.migrate()` function transfers Stability Pool deposits and unsold collateral tokens to a new strategy contract and updates the PSM pointer. However, it doesn't update the unsold collateral in the new PSMStrategy.

```
function migrate(address newStrategy) external override {
    _requireRole(Roles.MIGRATOR);
    // Withdraw COLLAR from SP and transfer to new strategy.
    address[] memory assets = STABILITY_POOL.getAssets();
    uint256 collarAmount = STABILITY_POOL.calcCompoundedDeposit(address(this));
    STABILITY_POOL.withdraw(collarAmount, newStrategy, address(this));
    // Transfer all unsold collateral to new strategy.
    uint256 assetNum = assets.length;
    uint256 unsold;
    for (uint256 i = 1; i < assetNum; ++i) {
        require(_activeAuctions[assets[i]].length() == 0,
            PSMStrategy_ActiveAuctionsExist(assets[i]));
        unsold = unsoldCollateral[assets[i]];
        if (unsold > 0) {
            unsoldCollateral[assets[i]] = 0;
            IERC20(assets[i]).safeTransfer(newStrategy, unsold);
        }
    }
    // Replace the strategy in the contracts.
    CallistoPSM(PSM).migrateToNewStrategy(newStrategy);
    emit Migrated(newStrategy);
}
```

With `unsoldCollateral` zeroed out and not updated in the new strategy, the new PSMStrategy will not be able to create auctions for the unsold collateral transferred during migration, leaving them sitting idle in the new contract without being utilized.

H-13 Migration Zeroes Unsold Collateral

CONTINUED

This concrete behaviour is bypassed in your integration/fork tests by the use of the `NewPSMStrategy` mock contract which implements a mock `finalizeMigration` function that updates the `unsoldCollateral` mapping in the new strategy.

— RECOMMENDATION

Consider adding a migration step to update the `unsoldCollateral` mapping in the new strategy contract during the migration process. Or introduce a single-use governance initializer callable straight after the pointer swap. This ensures that the new strategy is fully aware of the unsold collateral it holds, allowing it to manage and auction off these assets as intended.

— RESOLUTION

Status: Resolved.

MEDIUM **M-01**

RESOLVED

Auto-Rescinds Freeze

CATEGORY Unexpected Behavior
LOCATION OHMDelegationManager.sol
REVIEW Main Review

DESCRIPTION

`_autoRescindDelegations` walks the delegate list and calls `_rescindDelegation` with whatever amount is needed to reach the target undelegated balance. After subtracting the rescind amount,

`_rescindDelegation` immediately enforces that the remaining position still satisfies the global floor:

```
delegatedBalance -= rescindAmount;  
...  
require(  
  delegatedBalance >= minDelegationAmount,  
  OHMDelegationManager_BelowMinimumDelegationAmount(delegatedBalance, minDelegationAmount)  
);
```

All delegations were approved under the old `minDelegationAmount`. If governance later bumps this parameter, any delegate now sitting below the new threshold causes the require to trip during auto-rescind, and the whole withdrawal/closedown flow reverts. In a live scenario where the limit is tightened, stakers would be unable to withdraw collateral or autodelegate because every attempt to prune leftover delegations would hit this strict guard.

Impact: raising the minimum delegation bricks withdrawals for accounts with legacy delegates.

RECOMMENDATION

Skip the minimum-delegation guard when `_rescindDelegation` is reducing positions as part of an auto-rescind, or normalize legacy delegations before enforcing the new limit: for example, if `delegatedBalance < minDelegationAmount` after subtraction, delete the entry instead of reverting.

RESOLUTION

Status: Resolved.

M-01 Auto-Rescinds Freeze

CONTINUED

<https://github.com/CallistoDA0/callisto-core/pull/69>

MEDIUM **M-02**

RESOLVED

Wrong Debt Decimals Accounting

CATEGORY Math

LOCATION <https://github.com/GuardianOrg/callisto-core-team1-1759852427511/blob/main/src/policies/CallistoVault.sol#L306>

REVIEW Main Review

DESCRIPTION

When the vault borrows debts token from Cooler, the debt is saved as WAD, by calling `_updateWadDebtPrincipal`. However, all debt token interactions are done in terms of assets, when "investing" for example:

```
uint256 dTokenAmount = _convertWadToDebtToken(wadDebt);
strategy_.invest(dTokenAmount);
```

On the other hand, when sweeping profits, the vault deals with strategy interactions with the same asset unit as when calling `_updateWadDebtPrincipal`:

```
uint256 strategyProfit = strategy.profit();
if (strategyProfit != 0) strategy.divestProfit(amount, address(this));
// Only decrease debt if the amount exceeds strategy profit (i.e. returning principal).
if (amount > strategyProfit) _updateWadDebtPrincipal(-int256(FixedPointMathLib.rawSub(amount,
strategyProfit)));
```

This is fine for now, as the current debt token (USDS) is 18 decimals as well, but will break if the vault migrates to a token with non-18 decimals.

RECOMMENDATION

Consider converting the debt token amounts to WAD when calling `_updateWadDebtPrincipal` in `sweepProfit`.

RESOLUTION

Status: Resolved.

<https://github.com/CallistoDA0/callisto-core/pull/55>

MEDIUM **M-03**

RESOLVED

Arbitrary Swap Data

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/callisto-core-team1-1759852427511/blob/bd3ace38eb37e1703a1f3bacd266e8f305b97e4d/src/policies/CallistoVault.sol#L1132
REVIEW	Main Review

DESCRIPTION

The `processPendingDeposits` function can be called by anyone to process pending deposits. When the warm-up period is enabled, `OHM` must first be staked for a set duration using `OlympusStaking.stake()` and later claimed through `OlympusStaking.claim()`.

In this case, "the Callisto protocol can activate `OHMToG0HMMode.Swap`, allowing governance to set a custom `IOHMSwapper` through `setSwapMode`. The swapper contract can interact with supported exchanges and perform slippage or parameter validations as needed."

The problem is that `processPendingDeposits` is publicly callable and allows the caller to provide arbitrary `swapData`. A malicious user could craft `swapData` with manipulated parameters to influence the swap execution, causing value loss to the vault.

RECOMMENDATION

Restrict who can call `processPendingDeposits` or validate the provided `swapData` against trusted parameters to prevent arbitrary or unsafe swap execution.

RESOLUTION

Status: Resolved.

<https://github.com/CallistoDA0/callisto-core/pull/79>

MEDIUM **M-04**

RESOLVED

Warm-up Griefing

CATEGORY	DoS
LOCATION	https://github.com/GuardianOrg/callisto-core-team1-1759852427511/blob/bd3ace38eb37e1703a1f3bacd266e8f305b97e4d/src/policies/CallistoVault.sol#L1165
REVIEW	Main Review

DESCRIPTION

The `processPendingDeposits` function can be called by anyone to process pending deposits. When the warm-up period is active, OHM must first be staked for a set duration using `OlympusStaking.stake` and later claimed via `OlympusStaking.claim`.

The problem is that the function allows the caller to specify the `ohmAmount`. A malicious user can repeatedly call `processPendingDeposits` with a dust-sized amount, triggering a new warm-up cycle each time. Once this happens, the vault cannot call `processPendingDeposits` again until the previous stake's warm-up period has elapsed. This behaviour can be repeatedly exploited, effectively blocking the vault from processing its full pending deposits and delaying strategy investment indefinitely.

RECOMMENDATION

Modify `processPendingDeposits` to use the full pendingOHMDeposits balance when the warm-up period is active, or enforce a minimum to prevent users from initiating dust-sized stakes that block further processing.

RESOLUTION

Status: Resolved.

<https://github.com/CallistoDA0/callisto-core/pull/80>

MEDIUM **M-05**

RESOLVED

Unaccounted Interest

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/callisto-core-team1-1759852427511/blob/bd3ace38eb37e1703a1f3bacd266e8f305b97e4d/src/policies/CallistoVault.sol#L1033
REVIEW	Main Review

— DESCRIPTION

The profit function correctly reserves profit to cover outstanding interest reimbursements in the general `wadCurrent >= uint256(wadBorrowed)` and `wadCurrent < uint256(wadBorrowed)` cases. However, when `wadBorrowed` is negative or when `wadDebt == 0` under the first condition, the function returns `strategyProfit` directly without deducting `totalInterestClaimsWad`. This can result in profit being incorrectly swept to the treasury before reserving funds for user interest reimbursements, delaying payouts to users.

— RECOMMENDATION

Ensure `totalInterestClaimsWad` is consistently accounted for across all branches of the profit function so that adequate funds remain available to cover pending interest reimbursements.

— RESOLUTION

Status: Resolved.

<https://github.com/CallistoDAO/callisto-core/pull/91>

MEDIUM **M-06**

RESOLVED

Missing Liquidation Check In Execute

CATEGORY Access Control

LOCATION <https://github.com/GuardianOrg/callisto-core-team1-1759852427511/blob/bd3ace38eb37e1703a1f3bacd266e8f305b97e4d/src/policies/CallistoVault.sol#L267>

REVIEW Main Review

DESCRIPTION

The processPendingDeposits function includes the _requireVaultNotLiquidated check to prevent pending deposits from being processed if a vault has been liquidated. However, the execute function, which can also process deposits when the warm-up period is zero, does not implement this safeguard. Consequently, if a vault has been liquidated, the execute function could still process pendingOHMDeposits and invest to the strategy, potentially leading to inconsistent state or unintended fund allocation.

RECOMMENDATION

Add the _requireVaultNotLiquidated check to the execute function to ensure pending deposits aren't processed for liquidated vaults.

RESOLUTION

Status: Resolved.

<https://github.com/CallistoDAO/callisto-core/pull/41>

MEDIUM **M-07**

RESOLVED

Emergency Burn Skews Excess Accounting

CATEGORY Logical Error
LOCATION CallistoPSM.sol
REVIEW Main Review

DESCRIPTION

`burnCOLLARInEmergency` withdraws and burns COLLAR from the strategy but never adjusts `excessCOLLAR`:

```
function burnCOLLARInEmergency(uint256 amount) external override {  
  _requireRole(Roles.ADMIN);  
  _requireNonzeroAmount(amount);  
  _burnCOLLARInStrategy(PSMStrategy(strategy), amount);  
  emit COLLARBurnedInEmergency(amount);  
}
```

After the burn, `excessCOLLAR` still reflects the pre-burn surplus, so downstream logic believes there is residual excess even though the tokens are gone. `swapIn` computes `excess = excessCOLLAR - collarPendingBurning` and, expecting excess to exist, diverts incoming COLLAR into `collarPendingBurning` instead of redepositing into the strategy, starving liquidity. `burnExcessCOLLAR` observes a positive excess but finds no withdrawable COLLAR, leaving the counter elevated indefinitely. Therefore, accounting drifts out of sync and future inflows are misrouted until new COLLAR is minted.

RECOMMENDATION

Within the `burnCOLLARInEmergency` function, decrement `excessCOLLAR` by the burnt amount (saturating at zero) so state mirrors the actual balance. Consider re-evaluating related bookkeeping variables after emergency burns to keep `collarPendingBurning` and strategy inventory aligned.

RESOLUTION

Status: Resolved.

<https://github.com/CallistoDAO/callisto-core/pull/35>

MEDIUM **M-08**

ACKNOWLEDGED

Roles-dependent Contracts

CATEGORY Unexpected Behavior
LOCATION CallistoRoles.sol
REVIEW Main Review

DESCRIPTION

`RolesConsumer` stores the `ROLES` module address in an immutable field set at construction. Contracts that inherit it (e.g., `CallistoToken`, `PSMStrategy`, `CallistoTimelock`) call `_requireRole` against that fixed instance. When governance upgrades the `ROLES` module via `Kernel._upgradeModule`, policies using `PolicyRolesConsumer` are re-wired to the new module, but these standalone consumers keep hitting the old contract. Any grants or revocations performed on the fresh module never propagate to them, so a role revoked on the new `ROLES` deployment remains active for those contracts.

Example: after the upgrade, governance removes `Roles.MINTER` from an address on the new module. `CallistoToken.mint` still calls `_requireRole` on the stale module where the flag remains true, letting the attacker mint indefinitely. The issue persists until those consumer contracts are redeployed or reconfigured manually, bypassing role governance post-upgrade.

RECOMMENDATION

Refactor role-guarded contracts to fetch `ROLES` dynamically. Replace the immutable `ROLES` in `RolesConsumer` with a mutable reference set via the kernel (mirroring `PolicyRolesConsumer`) or add a kernel-only hook that updates the cached module during upgrades. Ensure `Kernel._upgradeModule` calls that hook so all consumers read from the current `ROLES` module before revocations are expected to take effect.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-09**

RESOLVED

repayDebtAndWithdrawColl Wrongly Reverts

CATEGORY Unexpected Behavior

LOCATION <https://github.com/GuardianOrg/callisto-core-team1-1759852427511/blob/main/src/external/cdp/BorrowerOperations.sol#L277>

REVIEW Main Review

DESCRIPTION

The `repayDebtAndWithdrawColl` function in `BorrowerOperations` always reverts when the system is in recovery mode and the user tries to withdraw collateral (`coll > 0`), even if the repayment would bring the system's Total Collateral Ratio (TCR) back above the Critical Collateral Ratio (CCR). This happens because `v.isRecoveryMode` is checked before the repayment is applied, causing `_requireNoCollWithdrawal(coll)` to revert immediately. As a result, users cannot atomically repay enough debt to restore system health and withdraw collateral in the same transaction.

RECOMMENDATION

Consider checking the recovery mode after the user's debt repayment is applied, so the function can allow collateral withdrawal if the system exits recovery as a result of the repayment.

PROOF OF CONCEPT <https://gist.github.com/Ali-Shehab/68b0daaffbe4d46718faf87e897e9f56>

RESOLUTION

Status: Resolved.

<https://github.com/CallistoDAO/callisto-core/pull/87>

MEDIUM **M-10**

RESOLVED

USDS Loss From Unclaimable Reimbursements

CATEGORY Unexpected Behavior
LOCATION CallistoVault.sol
REVIEW Main Review

DESCRIPTION

If a user repays the cooler's interest but there isn't enough USDS to distribute to their account, the unpaid amount is tracked as `interestClaimsWad` and added to `totalInterestClaimsWad`. When sufficient USDS becomes available, the user can claim these funds through the `claimInterestReimbursement` function.

However, if the vault is liquidated, calling `claimInterestReimbursement` will revert due to the `_validateAndCalculateClaimableWad` check.

In such cases, the admin can call `setEmergencyRedeem(true)` to enable the `emergencyRedeem` function. The `emergencyRedeem` function excludes `totalReimbursementClaims` from the strategy balance. As a result, users are unable to claim their pending interest reimbursements—since `claimInterestReimbursement` reverts—and these funds are not redeemable through `emergencyRedeem`, leaving them permanently locked in the vault strategy.

RECOMMENDATION

There are two possible solutions:

- If the intention is to prevent users from claiming reimbursements that are included in the strategy balance during liquidation, consider removing the check there and allow the full strategy balance to be withdrawn.
- If the intention is to reserve those funds for users who repaid protocol interest to the cooler, then remove this check from the claim function.

RESOLUTION

Status: Resolved.

<https://github.com/CallistoDAO/callisto-core/pull/90>

MEDIUM **M-11**

RESOLVED

Sub-share Withdrawals Steal Collateral

CATEGORY Rounding
LOCATION BorrowerOperations.sol
REVIEW Main Review

DESCRIPTION

`BorrowerOperations.withdrawColl` and its variants let a borrower request any coll amount, transfer that exact number of tokens out, and rely on `VesselManager` to keep the vessel's accounting in sync. In `_decreaseVesselAmounts` the withdrawal is converted into shares with `SharesMathLib.toSharesDown`:

```
uint256 collShares = SharesMathLib.toSharesDown(vc.collDecrease, state.collIndex);  
...  
vessel.collShares -= collShares;  
...  
newColl = SharesMathLib.toAssetsDown(vessel.collShares, state.collIndex);
```

When `collIndex` exceeds 1 (which happens after the first redistribution or liquidation), any withdrawal smaller than one share produces `collShares == 0`. The borrower still receives the ERC20 because `BorrowerOperations` always transfers the full coll:

```
vm.sendCollateral(asset, _msgSender(), coll);
```

Yet `vessel.collShares` never changes, so `vm.getVesselPosition` keeps reporting the original collateral and both ICR and NICR remain untouched. Repeating small withdrawals lets the borrower drain real collateral while the protocol still thinks it is fully backed, resulting in eventual insolvency. Note that the amount siphoned per call is the 'dust' created by the double `toSharesDown` -> `toAssetsDown` conversion: the actual loss equals `collDecrease - toAssetsDown(toSharesDown(collDecrease, collIndex), collIndex)`. As soon as liquidations or redistributions lift `collIndex` above 1, even a 1-wei withdrawal can route one full collateral unit out of the pool. After major liquidation events that push the index higher, the exploit becomes dramatically more profitable because the unburned shares represent larger and larger real balances.

RECOMMENDATION

M-11 Sub-share Withdrawals Steal Collateral

CONTINUED

Convert the requested collateral to shares with a rounding-up operation (or explicitly clamp the withdrawal to `toAssetsDown` of the burned shares) so that every transfer decrements `vessel.collShares` and the reported collateral.

PROOF OF CONCEPT <https://gist.github.com/r0bert-ethack/2fc9f67784c24811517f73d7e0576b5b>

— RESOLUTION

Status: Resolved.

<https://github.com/CallistoDAO/callisto-core/pull/86>

MEDIUM **M-12**

RESOLVED

Unaccrued Interest In `getRedemptionHints`

CATEGORY Math

LOCATION <https://github.com/GuardianOrg/callisto-core-team1-1759852427511/blob/main/src/external/cdp/VesselManager.sol#L246>

REVIEW Main Review

DESCRIPTION

`getRedemptionHints` calculates hints for efficient redemption operations and returns the parameters to pass to `redeemCollateral`. One of these is `partialRedemptionHintNewNICR`, which is later validated in `redeemCollateral`:

```
(, totals.newDebt, totals.newNICR) = _rebalance(asset, currentBorrower, price, totals.collLot, totals.debtLot);  
if (FixedPointMathLib.dist(partialRedemptionHintNewNICR, totals.newNICR) > 5e14) break;
```

The calculation of `partialRedemptionHintNewNICR` uses `debtIndexBelow0LTV` without accruing interest, while `redeemCollateral` does consider accrued interest. When interest is accrued, `debtIndexBelow0LTV` increases.

This discrepancy can make `partialRedemptionHintNewNICR` consistently lower than the actual NICR computed in `redeemCollateral`, causing partial redemptions to fail.

RECOMMENDATION

Preview interest and use the previewed index instead of the cached one in `collDebtState`. Replace:

```
uint256 indexBelow = collDebtState[asset].debtIndexBelow0LTV;
```

with:

```
(,, uint256 indexBelow,,) = _calculateInterest(collDebtState[asset]);
```

RESOLUTION

Status: Resolved.

<https://github.com/CallistoDAO/callisto-core/pull/88/files>

MEDIUM **M-13**

RESOLVED

USDS Profit Locked If Vault Is Liquidated

CATEGORY Logical Error
LOCATION CallistoVault.sol
REVIEW Main Review

DESCRIPTION

When the vault enters liquidation mode, users are expected to withdraw their remaining share of assets through `emergencyRedeem`.

However, `emergencyRedeem` computes withdrawable funds using `totalAssetsAvailable`:

```
function totalAssetsAvailable() public view returns (uint256) {  
    int256 principalAssets_ = principalAssets;  
    if (principalAssets_ < 0) return 0;  
    return Math.min(uint256(principalAssets_), yieldVault.maxWithdraw(address(this)));  
}
```

This function does not include the profit, meaning the yield strategy's accumulated profit is never withdrawn.

Furthermore, if the admin decides to sweep the funds to the treasury, the `profit()` function contains the following check:

```
if (isVaultPositionLiquidated()) return 0;
```

As a result, it will always return 0 if the vault is liquidated, leaving those profits permanently locked in the vault strategy.

RECOMMENDATION

Consider either allowing the treasury to withdraw the profit through allowing it in `sweepProfit` or distributing it to emergency redeemers through adding the profit to `totalAssetsAvailable` in `emergencyRedeem`.

RESOLUTION

Status: Resolved.

<https://github.com/CallistoDAO/callisto-core/pull/68>

MEDIUM **M-14**

RESOLVED

Floor Check Dilutes Depositor Rewards

CATEGORY Validation

LOCATION <https://github.com/GuardianOrg/callisto-core-team1-1759852427511/blob/main/src/external/cdp/StabilityPool.sol#L629-L637>

REVIEW Main Review

DESCRIPTION

When an offset amount is less than total deposits, the product is recalculated to deplete deposits. If the product shifts enough, `scale` is incremented by 1:

```
if (scaledP < SCALE_FACTOR) {  
    /* If multiplying the product by a non-zero product factor would reduce the product below  
    * the scale boundary, increment the scale.  
    */  
    product_ = (rawP * SCALE_FACTOR) / Constants.PERCENTAGE_PRECISION;  
    ++scale_  
    scale = scale_  
    emit ScaleUpdated(scale_);  
}
```

When users withdraw their deposits, the deposit is recomputed to account for product and scale changes in `_calcCompoundedDeposit`:

```
compoundedDeposit = (initialDeposit * product_) / pSnapshot / SCALE_FACTOR;
```

After that, the code checks if the deposit is less than a billionth of the initial deposit and, if so, discards it. The comment says this was originally to ensure withdrawals favor the pool, but error corrections now handle rounding:

```
/*  
    * If a compounded deposit is less than a billionth of the initial deposit, return 0.  
    *  
    * Note. Originally, this line was in place to stop rounding errors making the deposit too  
    large.  
    * However, the error corrections should ensure the error in product "favors the pool", i.e.  
    * any given compounded deposit should be slightly less than its theoretical value.  
    * So, it is unclear whether this line is still really needed.
```

M-14 Floor Check Dilutes Depositor Rewards

CONTINUED

```
*/
```

```
if (compoundedDeposit < initialDeposit / SCALE_FACTOR) return 0;
```

This is problematic because discarded deposits are still counted in `totalDeposits` but cannot be claimed by anyone. As a result, rewards distributed to depositors are diluted by the amount of deposits that were thrown away, and would accumulate over time as more deposits are discarded.

— RECOMMENDATION

Consider removing the check that discards small deposits in `_calcCompoundedDeposit`.

PROOF OF CONCEPT <https://gist.github.com/Ali-Shehab/2c2b971f31399a229d852e355a065f8f>

— RESOLUTION

Status: Resolved.

<https://github.com/CallistoDA0/callisto-core/pull/89>

MEDIUM **M-15**

RESOLVED

Zero Heart Keeper Rewards

CATEGORY Logical Error
LOCATION CallistoHeart.sol
REVIEW Main Review

DESCRIPTION

In `CallistoHeart.beat` the contract updates `lastBeat` before it calculates the keeper reward. At the `CallistoHeart` contract the code executes `lastBeat = currentTime - ((currentTime - lastBeatTime) % freq);` and only after that, it calls `currentReward()`. Inside `currentReward` the first operation is `uint48 nextBeat = lastBeat + freq;`. Because `lastBeat` has already been advanced to the most recent schedule slot, `nextBeat` ends up greater than or equal to `currentTime`, so the guard `if (currentTime <= nextBeat) return 0;` immediately triggers. Every call to beat therefore returns zero reward regardless of delay, removing the keeper incentive. Without rewards, off-chain automation has no economic reason to run beat, threatening to halt vault/PSM upkeep that relies on this call.

RECOMMENDATION

Calculate the reward using the pre-update timestamp, for example by storing `lastBeatTime` in memory and passing it to `currentReward` before assignment, or by moving the `lastBeat = ...` line to after reward minting so `currentReward` bases the auction on the prior beat.

RESOLUTION

Status: Resolved.

<https://github.com/CallistoDA0/callisto-core/pull/34>

MEDIUM **M-16**

RESOLVED

Loss-rate Increment Can Block Liquidations

CATEGORY Math

LOCATION <https://github.com/GuardianOrg/callisto-core-team1-1759852427511/blob/main/src/external/cdp/StabilityPool.sol#L698>

REVIEW Main Review

DESCRIPTION

When offsetting debt from the Stability Pool, the pool checks if the offset debt equals the entire total deposit. If so, `scale` and `product` are reset and the `epoch` is incremented to deplete the pool and deposits:

```
++epoch;
delete scale;
product = Constants.PERCENTAGE_PRECISION;
```

However, when the offset debt is less than the total deposits, deposits (and total deposits) are reduced by a rate derived from the offset amount. This rate, `debtTokenLossRate`, is then used to compute the new product (which scales down deposits):

```
uint256 newProductFactor = Constants.PERCENTAGE_PRECISION - debtTokenLossRate;
uint256 rawP = product * newProductFactor;
uint256 scaledP = rawP / Constants.PERCENTAGE_PRECISION;
// ... snip ...
product = scaledP;
```

When calculating the loss rate, the implementation increments the rate by 1 to "favor the pool":

```
if (debtTokenLoss) {
    /* Add 1 to make the error in the quotient positive and the debtToken loss "a little too much"
    to ensure
    * the error in any given compounded deposit favors the stability pool.
    */
    ++rate;
    return (rate, rate * totalDeposits_ - numerator);
}
```

If the offset debt is exactly `totalDeposits - X` (with small `X`), this increment can push the rate to 100%, making `newProductFactor` zero and causing a revert:

M-16 Loss-rate Increment Can Block Liquidations

CONTINUED

```
require(product_ != 0, ZeroProduct());
```

This ultimately blocks liquidations when the offset debt is very close to total deposits.

— RECOMMENDATION

Consider refactoring the rate incremental logic to only do so if the rate isn't 99%, similar to:

```
if (rate < Constants.PERCENTAGE_PRECISION - 1) {  
  ++rate;  
  return (rate, rate * totalDeposits_ - numerator);  
} else {  
  return (rate, 0);  
}
```

PROOF OF CONCEPT <https://gist.github.com/Ali-Shehab/25547ea6239b56d748faec9996de4ef6>

— RESOLUTION

Status: Resolved.

<https://github.com/CallistoDAO/callisto-core/pull/85>

MEDIUM **M-17**

ACKNOWLEDGED

Cooler Principal Ledger Subtracts Interest

CATEGORY Logical Error
LOCATION CallistoVault.sol
REVIEW Main Review

DESCRIPTION

Repay paths forward the entire Cooler payment (principal + interest) into the principal tracker:

```
uint256 repaidInWad =  
OLYMPUS_COOLER.repay({ repayAmountInWad: uint128(currentDebtWad), onBehalfOf: address(this)  
});  
_updateWadDebtPrincipal(-int256(repaidInWad));  
uint128 amountRepaidInWad =  
OLYMPUS_COOLER.repay({ repayAmountInWad: wadDebt, onBehalfOf: address(this) });  
_updateWadDebtPrincipal(-int256(uint256(amountRepaidInWad)));
```

Borrowing, however, adds only the principal that was just drawn:

```
uint128 wadDebt = OLYMPUS_COOLER.borrow({ borrowAmountInWad: type(uint128).max, onBehalfOf:  
address(this), recipient: address(strategy_) });  
_updateWadDebtPrincipal(int256(uint256(wadDebt)));
```

Because `_updateWadDebtPrincipal` subtracts interest as well as principal, the ledger drifts below the true principal after every interest-bearing repayment. `_coolerInterest()` then treats the shortfall as if additional interest were owed:

```
int256 wadBorrowed = wadDebtPrincipal;  
uint256 wadCurrent = OLYMPUS_COOLER.accountDebt(address(this));  
wadDebt =  
(wadCurrent >= uint256(wadBorrowed))  
? FixedPointMathLib.rawSub(wadCurrent, uint256(wadBorrowed))  
: 0;
```

Once those phantom debts accumulate, users are forced to settle “interest” that no longer exists and the inflated numbers feed other flows (e.g. withdrawal debt checks), aggravating the risk of locked funds.

M-17 Cooler Principal Ledger Subtracts Interest

CONTINUED

— RECOMMENDATION

Clamp the principal tracker to principal only. When repaying, subtract at most the outstanding principal portion, never the accrued interest, and only zero it out when the Cooler position is fully closed. One approach is to compare `repaidInWad` against the current principal and subtract `Math.min(repaidInWad, principal)` while keeping a separate counter for interest.

PROOF OF CONCEPT <https://gist.github.com/r0bert-ethack/5b99e2ef002e198c117e43a56a3ce143>

— RESOLUTION

Status: Acknowledged.

LOW **L-01**

ACKNOWLEDGED

Deactivated Policies Keep Module Access

CATEGORY Unexpected Behavior
LOCATION Kernel.sol
REVIEW Main Review

DESCRIPTION

`Kernel._deactivatePolicy` simply re-queries the policy for permissions and revokes only the selectors it returns. The call uses `policy_.requestPermissions()`, which is a view hook under the policy's control. A malicious policy can change its internal configuration so that, when deactivation is requested, `requestPermissions()` yields an empty or truncated array. `_setPolicyPermissions` then touches nothing, leaving the original `modulePermissions` entries untouched. Module guards consult only `modulePermissions[keycode][policy][selector]` and never check `isPolicyActive`, so the deactivated policy continues to call privileged module functions even after removal. Therefore, a compromised policy can refuse to relinquish access and retain full control over modules indefinitely.

RECOMMENDATION

Record the exact permission set granted at activation and reuse that stored list during deactivation. For example, persist `Permissions[]` in kernel storage keyed by policy and clear those entries on `_deactivatePolicy`, or iterate through all modules/selectors and reset the bits without relying on a policy-supplied list.

RESOLUTION

Status: Acknowledged.

LOW **L-02**

RESOLVED

Strategy Migration Misses Yield Vault Check

CATEGORY Validation
LOCATION VaultStrategy.sol
REVIEW Main Review

DESCRIPTION

`VaultStrategy.migrate` hands its entire `yieldVault` share balance to the replacement strategy without proving they target the same vault. The vault only enforces that both strategies reference the same debt token and vault address during `migrateToNewStrategy` but never checks `newStrategy.yieldVault()`. If the new deployment was configured with a different underlying yield vault, the received shares are meaningless: the new strategy cannot redeem them, yet the old strategy deleted its own state and permissions. The assets backing those shares remain trapped in the original vault, permanently reducing recoverable capital.

RECOMMENDATION

Persist the current `yieldVault` address during activation and `require` `VaultStrategy(newStrategy).yieldVault() == yieldVault` before transferring shares. Alternatively, redeem all shares to underlying assets in the legacy strategy, then pass raw assets to the new strategy for redeposit. This avoids any dependency on matching share types.

RESOLUTION

Status: Resolved.

<https://github.com/CallistoDA0/callisto-core/pull/92>

LOW **L-03**

RESOLVED

Timelock Delay Can Be Set To Zero

CATEGORY Warning
LOCATION CallistoTimelock.sol
REVIEW Main Review

DESCRIPTION

`CallistoTimelock.updateDelay` is the only path that mutates `_minDelay`, and it is callable once the timelock schedules a self-targeted operation. The code simply emits `MinDelayChange` and assigns the caller-provided value without validating it. Although the constructor enforces a 1–3 day bound, a proposer can queue an `updateDelay(0)` operation respecting the current delay; when it executes, `_minDelay` becomes zero. From that moment every privileged action the timelock controls can be scheduled and executed in the same transaction, removing the intended governance safety window.

RECOMMENDATION

Inside `updateDelay`, require the new delay to satisfy the same bounds as the constructor, for example `require(newDelay >= MIN_DELAY_BOUND && newDelay <= MAX_DELAY_BOUND, TimelockInvalidDelay());` before committing the state change. If different limits are desired, enforce those invariants there so self-updates cannot disable or shrink the timelock without a bounded delay.

RESOLUTION

Status: Resolved.

<https://github.com/CallistoDAO/callisto-core/pull/49>

LOW **L-04**

RESOLVED

Collateral Withdraw Favors Users

CATEGORY	Rounding
LOCATION	https://github.com/GuardianOrg/callisto-core-team1-1759852427511/blob/main/src/external/cdp/VesselManager.sol#L1077
REVIEW	Main Review

DESCRIPTION

In `_decreaseVesselAmounts`, collateral decreases convert assets → shares with `toSharesDown`:

```
uint256 collShares = SharesMathLib.toSharesDown(vc.collDecrease, state.collIndex);
```

However, this favors the users instead of the protocol. This allows the user to always burn fewer shares than expected.

RECOMMENDATION

Consider rounding the conversion in favor of the protocol, e.g., by using `toSharesUp` instead and capping the shares to burn at the vessel's current shares:

```
uint256 collShares = FixedPointMathLib.min(SharesMathLib.toSharesDown(vc.collDecrease, state.collIndex), vessel.collShares);
```

RESOLUTION

Status: Resolved.

<https://github.com/CallistoDAO/callisto-core/pull/99>

LOW **L-05**

RESOLVED

Setup Launches With Zero Borrow Fee

CATEGORY Configuration
LOCATION AdminContract.sol
REVIEW Main Review

DESCRIPTION

`AdminContract._addNewCollateral` seeds every new collateral with `BORROWING_FEE_DEFAULT`, and `setup()` calls it before activating the first asset. The constant is currently 0 ether in the `AdminContract`, even though the inline comment says 0.5%. As soon as setup finishes, the asset is live and `BorrowerOperations._computeNetDebt` reads a zero fee, so anyone opening a vessel before the timelock queues and executes a `setBorrowingFee` update can borrow without paying the intended 0.5% treasury revenue. Because the timelock delay is unavoidable, opportunistic users can front-run the fee change and launch with fee-free borrowing, draining early protocol income.

RECOMMENDATION

Set `BORROWING_FEE_DEFAULT` to the intended 0.5% or delay collateral activation until after governance applies the correct fee through the timelock.

RESOLUTION

Status: Resolved.

<https://github.com/CallistoDAO/callisto-core/pull/48>

LOW **L-06**

RESOLVED

Disabled Markets Still Mint Debt

CATEGORY	Unexpected Behavior
LOCATION	AdminContract.sol; BorrowingOperations.sol
REVIEW	Main Review

DESCRIPTION

`AdminContract.setIsActive(asset, false)` flips the collateral's active flag, and `BorrowingOperations.openVessel` honours it, but the adjustment path `_activeVesselCheck`, invoked by `borrowDebt`, `addCollAndBorrowDebt` and other mutators at `BorrowingOperations`, never revalidates `admin.getIsActive(asset)`. Once a borrower has an active vessel they can wait for governance to disable the market, then call `borrowDebt` to mint fresh COLLAR because the Admin flag is ignored. `_activeVesselCheck` still reads prices, CCR/MCR and vessel state, so operations succeed and `VesselManager.increaseVesselAmounts` happily records the new debt. The emergency kill switch therefore fails: hostile holders keep inflating supply and draining collateral after governance attempts to freeze the market.

RECOMMENDATION

Reintroduce the `getIsActive` gate for all adjustment paths (e.g. inside `_activeVesselCheck` before returning) or enforce it down in `VesselManager.increaseVesselAmounts` so inactive markets cannot mint or adjust debt.

RESOLUTION

Status: Resolved.

<https://github.com/CallistoDAO/callisto-core/pull/78>

LOW **L-07**

RESOLVED

Missing Debt Token Migration Check

CATEGORY	Unexpected Behavior
LOCATION	VaultStrategy.sol
REVIEW	Main Review

DESCRIPTION

During a debt token migration the migrator calculates how many assets to move using the old ERC4626 vault:

```
uint256 assets = IERC4626(strategy.yieldVault()).maxWithdraw(address(strategy));
```

It then calls:

```
strategy.migrateAsset(newDebtToken, newYieldVault, msg.sender, assets,  
principalAssetsConverted);
```

Inside `VaultStrategy.migrateAsset` the strategy redeems only the current `maxRedeem` amount:

```
uint256 shares = yieldVault_.maxRedeem(address(this));  
uint256 redeemed = yieldVault_.redeem(shares, receiver, address(this));  
require(redeemed == assets, VaultStrategy_InvalidRedeem(assets, redeemed));  
asset.forceApprove(address(yieldVault), 0);  
asset = newDebtToken;  
yieldVault = newYieldVault;
```

If the old ERC4626 enforces withdrawal throttles, returning, for example, 5% via `maxWithdraw` / `maxRedeem`, this equality still passes (both are the throttled amount), but most shares remain in the old vault. Immediately after the redeem the strategy wipes its allowance and points `asset` / `yieldVault` at the new vault, so the residual balance is orphaned and locked forever. Migration apparently succeeds, yet principal is stuck and lost.

RECOMMENDATION

Before updating `asset` / `yieldVault`, assert that the old share balance is zero. A simple guard such as:

```
require(yieldVault.balanceOf(address(this)) == 0, "migration incomplete");
```

L-07 Missing Debt Token Migration CheckCONTINUED

— RESOLUTION

Status: Resolved.

<https://github.com/CallistoDA0/callisto-core/pull/77>

LOW **L-08**

RESOLVED

Broken Functionality In borrowDebt

CATEGORY	Logical Error
LOCATION	BorrowerOperations.sol
REVIEW	Main Review

DESCRIPTION

In the `borrowDebt` function, when a user wants to borrow, multiple checks are performed depending on whether the system is in recovery mode. If it is in recovery mode, the function checks whether the ICR is above the CCR and whether the new ICR is higher than the old ICR. However, since the `v.price` used to compute both ICRs is the same and the user is only increasing their debt, the `borrowDebt` function will always revert in recovery mode.

RECOMMENDATION

Consider always reverting if in recovery mode without any unneeded checks, if the intention is to block adding debts when in recovery mode

RESOLUTION

Status: Resolved.

<https://github.com/CallistoDAO/callisto-core/pull/45>

LOW **L-09**

RESOLVED

getGainsOf Miscalculates Pending Interest

CATEGORY Math

LOCATION <https://github.com/GuardianOrg/callisto-core-team1-1759852427511/blob/main/src/external/cdp/StabilityPool.sol#L541-L557>

REVIEW Main Review

DESCRIPTION

When users claim their gains using `claimGains`, it first accrues any pending interest via `_accrueInterestAndUpdateSummary`, then accrues the account's gains via `_accrueAssetGainsFor`, ensuring the user claims everything.

On the other hand, users can call `getGainsOf` to view pending gains without claiming them. However, `getGainsOf` does not accrue pending interest; instead, it tries to estimate it using:

```
uint256 pendingInterest =
DebtToken(debtToken).balanceOf(address(this)) - distributedInterest - totalDeposits_;
if (pendingInterest > 0 && totalDeposits_ > 0) {
// Calculate additional gains: Initial deposit * Additional sum from pending interest / P
snapshot.
gains[_REWARD_TOKEN_POSITION] += (
initialDeposit * (pendingInterest * Constants.PERCENTAGE_PRECISION / totalDeposits_ *
product)
) / pSnapshot / Constants.PERCENTAGE_PRECISION;
}
```

This diverges from the claiming path and leads to miscalculation when the scale diff > 0 (after a large offset).

As a result, `getGainsOf` can report incorrect rewards.

RECOMMENDATION

Consider replacing the whole pending interest calculation in `getGainsOf` with something similar to:

```
gains[_REWARD_TOKEN_POSITION] = rDepositorGains[_REWARD_TOKEN_POSITION];
if (rSums[0] != 0) {
(uint256 assetGainRate,) = _calcRateAndError(
false,
```

L-09 getGainsOf Miscomputes Pending Interest

CONTINUED

```
DebtToken(debtToken).balanceOf(address(this)) - distributedInterest - totalDeposits,  
totalDeposits,  
gainOffsetErrors[_REWARD_TOKEN_POSITION]  
);  
uint256 rSums0 = rSums[0];  
uint256 rNextSums0 = rNextSums[0];  
if (scaleSnapshot == scale) rSums0 += assetGainRate * product;  
else rNextSums0 += assetGainRate * product;  
gains[_REWARD_TOKEN_POSITION] +=  
_calcNewGain(initialDeposit, pSnapshot, rSums0, rNextSums0, rDepositorSums[0]);  
}
```

PROOF OF CONCEPT <https://gist.github.com/Ali-Shehab/8dec3296c9004790ff7a861776538d9e>

— RESOLUTION

Status: Resolved.

<https://github.com/CallistoDAO/callisto-core/pull/100>

LOW **L-10**

RESOLVED

Finite Router Approvals Can Exhaust

CATEGORY Best Practices

LOCATION <https://github.com/GuardianOrg/callisto-core-team1-1759852427511/blob/main/src/external/CDPRouter.sol#L46-L47>

REVIEW Main Review

DESCRIPTION

`CDPRouter` sets allowances to `type(uint128).max`:

```
ohm.safeApprove(vault, type(uint128).max);
```

```
vault.safeApprove(borrowerOperations, type(uint128).max);
```

ERC-20 allowances are `uint256`. Using `uint128` isn't "infinite" and can deplete with repeated transfers/withdrawals with time.

RECOMMENDATION

Consider using `type(uint256).max` for truly infinite allowances.

RESOLUTION

Status: Resolved.

<https://github.com/CallistoDA0/callisto-core/pull/67>

LOW **L-11**

RESOLVED

deletedToBalance Reverts

CATEGORY Best Practices
LOCATION OHMDelegationManager.sol
REVIEW Main Review

DESCRIPTION

`OHMDelegationManager.deletedToBalance` reads from an `EnumerableMap` via `.get`. OpenZeppelin's `EnumerableMap.get` reverts when the key is missing, so asking for the balance delegated to an address that has never been recorded throws instead of returning 0. That makes the view helper unusable for UIs or off-chain scripts unless they already know the delegate exists and contradicts the typical "balance query returns zero if unset" expectation.

RECOMMENDATION

Use `tryGet` (or first check `contains`) and return 0 when `deletedToAccount` has never been recorded. This keeps the helper read-only and ensures callers can safely probe delegation balances without pre-checking the map.

RESOLUTION

Status: Resolved.

<https://github.com/CallistoDA0/callisto-core/pull/66>

INFORMATIONAL **I-01**

ACKNOWLEDGED

Manager Sweep Can Drain Principal

CATEGORY Trust Assumptions
LOCATION CallistoVault.sol
REVIEW Main Review

DESCRIPTION

`CallistoVault.sweepProfit` trusts the internal `profit()` ledger to cap withdrawals even though only the manager/admin role can invoke it. Repeated manager-controlled cycles of `withdrawAllCollateralToPendingOHM` followed by `_handleDepositsToStrategy` shrink `wadDebtPrincipal` without clearing the Cooler loan. Once `wadDebtPrincipal` dips below zero, `profit()` short-circuits and exposes the entire strategy balance as surplus. `sweepProfit` then lets the same manager pull those funds to TRSRY, even updating the debt ledger downward without sending repayment on-chain:

```
if (amount > strategyProfit) {  
  _updateWadDebtPrincipal(-int256(FixedPointMathLib.rawSub(amount, strategyProfit)));  
}  
debtToken.safeTransfer(TRSRY, amount);
```

Because the Cooler liability remains outstanding, depositors are left with an insolvent vault while the treasury receives borrowed principal.

RECOMMENDATION

Restore accounting so that principal ledgers cannot drift below the real Cooler debt. When repaying, split interest from principal and only deduct the true principal component from `wadDebtPrincipal`, forcing actual on-chain repayment before any sweep exceeds the accrued `strategy.profit()`. Additionally, gate `sweepProfit` calls on verified external debt balances (e.g., require `accountDebt` to be fully covered) so treasury withdrawals cannot proceed while liabilities remain.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-02

ACKNOWLEDGED

Flashloan Guard Blocks Cross-calls Only

CATEGORY	Warning
LOCATION	StabilityPool.sol; TransientFlashloanGuard.sol
REVIEW	Main Review

DESCRIPTION

`TransientFlashloanGuard` simply flips a transient flag inside `enableFlashloanGuard` and lets `prohibitFlashloan` revert if that flag is set. This protects “other” entry points. For example, `StabilityPool.deposit` runs with `enableFlashloanGuard`, so any attempt to re-enter `StabilityPool.withdraw` in the same tx hits the guard and reverts. The modifier intentionally does not block re-entering the same function; you can call `deposit` from an ERC20 hook and execute it twice in one transaction. That behavior is expected, but developers might assume the guard enforces single-entry semantics like a traditional reentrancy lock and overlook the need for additional checks on self-reentry in future functions.

RECOMMENDATION

Do not rely on `enableFlashloanGuard` to block a function from re-entering itself. If a new entry point requires single-entry semantics, add an explicit reentrancy guard (e.g., OpenZeppelin’s `nonReentrant`) or a dedicated `require(!_flashloanGuard)` check in that function.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-03

RESOLVED

Treasury Withdrawal Event Mislabeled Actor

CATEGORY Warning
LOCATION TRSRYv1.sol
REVIEW Main Review

DESCRIPTION

TRSRYv1 defines `event Withdrawal(address indexed policy_, address indexed withdrawer, IERC20 indexed token, uint256 amount);`, where the second indexed field is meant to identify the entity exercising the withdrawal approval. In `CallistoTreasury.withdrawReserves` the code emits `Withdrawal(msg.sender, to, token, amount)`. That substitutes the recipient address for the withdrawer, so downstream monitoring sees the beneficiary as the actor and loses visibility into which policy actually pulled funds.

RECOMMENDATION

Emit the caller for both `policy_` and `withdrawer`, e.g. `emit Withdrawal(msg.sender, msg.sender, token, amount);`, or adjust the event definition to include an explicit recipient field so log consumers can differentiate between policy caller and destination.

RESOLUTION

Status: Resolved.

<https://github.com/CallistoDA0/callisto-core/pull/82>

INFORMATIONAL I-04

RESOLVED

Heart Beat Bricks On Warmup Toggle

CATEGORY	Unexpected Behavior
LOCATION	CallistoVault.sol; CallistoHeart.sol
REVIEW	Main Review

DESCRIPTION

`CallistoHeart.beat()` always invokes the vault's automation hook:

```
IExecutableByHeart executable = vault;  
if (address(executable) != address(0)) executable.execute();
```

While `ohmToGOHMMode` remains `ZeroWarmup`, `CallistoVault.execute()` only calls `_processPendingDepositsZeroWarmup`:

```
function execute() external override {  
  _requireRole(Roles.HEART);  
  uint256 ohmAmount = pendingOHMDeposits;  
  if (ohmAmount == 0) return;  
  if (ohmToGOHMMode == ICallistoVault.OHMTToGOHMMode.ZeroWarmup) {  
    _processPendingDepositsZeroWarmup(ohmAmount);  
  }  
}
```

That helper enforces a zero warm-up at runtime:

```
function _processPendingDepositsZeroWarmup(uint256 ohmAmount) private {  
  _validateAndUpdatePendingDeposits(ohmAmount);  
  _requireZeroWarmupPeriod();  
  ...  
}
```

If Olympus staking flips `warmupPeriod()` above zero while the vault is still in zero-warmup mode and there are queued deposits, `_requireZeroWarmupPeriod()` reverts. The revert bubbles back to `beat()`, so the Heart stops processing pending deposits and never reaches the PSM or reward code. Admins can manually recover by switching to `ActiveWarmup` or `Swap` mode, but until they intervene the automation loop is stuck and keepers cannot claim rewards.

I-04 Heart Beat Bricks On Warmup Toggle

CONTINUED

RECOMMENDATION

Have `CallistoVault.execute` (or the Heart) detect a non-zero warm-up and either dispatch to the active-warmup handler or skip the vault leg while surfacing an alert. Adding a guard such as `if (OLYMPUS_STAKING.warmupPeriod() > 0) return;` before `_processPendingDepositsZeroWarmup` keeps beats running and gives operators time to flip modes manually.

RESOLUTION

Status: Resolved.

INFORMATIONAL I-05

RESOLVED

DoS If Yield Vault Rejects Deposits

CATEGORY	Warning
LOCATION	CallistoVault.sol
REVIEW	Main Review

DESCRIPTION

Every heartbeat pipelines pending OHM through the vault's strategy:

```
// CallistoHeart.beat()
IExecutableByHeart executable = vault;
if (address(executable) != address(0)) executable.execute();
```

In zero-warmup mode the vault pushes directly into `_handleDepositsToStrategy`, which borrows debt tokens and immediately calls:

```
strategy_.invest(dTokenAmount);
VaultStrategy.invest just forwards the call to the external ERC4626 yield vault:
uint256 shares = yieldVault.deposit(assets, address(this));
```

There is no guard or fallback. Today sUSDS has no cap, but if that ERC4626 ever sets

`maxDeposit(address(this)) < assets`, whether via a future upgrade, governance limit, or simply because another tenant saturates capacity, deposit reverts with `ERC4626ExceededMaxDeposit`. Because Heart does not catch the revert, the entire `beat()` fails: pending OHM remains stuck, no gOHM is produced, PSM cleanup is skipped and keepers cannot earn rewards. Even though no cap is enforced at the moment, the automation pipeline is brittle against any future limit or third-party saturation of the shared vault.

RECOMMENDATION

Before calling `yieldVault.deposit`, check the allowance (e.g. `uint256 cap = yieldVault.maxDeposit(address(this));`) and only deposit up to the permitted amount, carrying over the remainder for the next beat. Alternatively wrap the deposit in a try/catch and surface an alert so Heart can continue processing other tasks instead of halting entirely.

RESOLUTION

Status: Resolved.

I-05 DoS If Yield Vault Rejects Deposits

CONTINUED

<https://github.com/CallistoDA0/callisto-core/pull/81>

INFORMATIONAL I-06

RESOLVED

Uninitialized Asset Updates Cause Event Noise

CATEGORY Events

LOCATION <https://github.com/GuardianOrg/callisto-core-team1-1759852427511/blob/bd3ace38eb37e1703a1f3bacd266e8f305b97e4d/src/external/cdp/VesselManager.sol#L576>

REVIEW Main Review

DESCRIPTION

Calling `VesselManager.accrueInterest(asset)` for an asset that has not been added via `addCollateralType` does not revert. Because the `collDebtState[asset]` struct is zero-initialized, `_calculateInterest` computes zero deltas (old indices are 0, so `indexUp(0, ...)` $\rightarrow$ 0 and the index deltas are 0). As a result, accrued interest is 0, `_mintInterest(0)` mints nothing, but `_accrueInterest` still updates `interestUpdatedAt` to `block.timestamp` and emits `InterestAccrued(asset, 0, 0, 0, 0)`. Functionally this is harmless, but it can be misleading for monitoring/analytics (appears as if interest accrual ran on that asset) and slightly muddies state (timestamp set) before the asset is actually configured, if monitoring only expect one asset from `vesselManager` and don't check which one is updated, this could be even more harmful as they will assume the new index/interest for the asset is 0.

RECOMMENDATION

Add this line in `_accrueInterest()`

```
function _accrueInterest(address asset) private returns (uint256 interestDebt) {
    CollDebtState storage state = collDebtState[asset];
    // Asset must be added/configured first.
    require(state.interestUpdatedAt != 0, VM_AssetNotConfigured());
    ....
}
```

RESOLUTION

Status: Resolved.

<https://github.com/CallistoDAO/callisto-core/pull/44>

INFORMATIONAL **I-07**

RESOLVED

Useless Check In addColl

CATEGORY	Best Practices
LOCATION	BorrowerOperations.sol
REVIEW	Main Review

DESCRIPTION

When users want to add collateral to a position, if the system is in recovery mode, it allows you to add collateral without any checks, since the system always wants to encourage adding collateral to move out of recovery mode. However, if the system is not in recovery mode, it checks that after adding the collateral, the new `TCR` remains above the `CCR`. This check is redundant because adding collateral can only increase the `TCR`, and since we are already in normal mode, the `TCR` is by definition greater than the `CCR`.

RECOMMENDATION

Remove the `_requireNewTCRIsAboveCCR` in `addColl` function.

RESOLUTION

Status: Resolved.

<https://github.com/CallistoDAO/callisto-core/pull/43>

INFORMATIONAL **I-08**

RESOLVED

Consider Using Or Removing The Unused Errors

CATEGORY Informational

LOCATION IDebtToken.sol / IStabilityPool.sol / IVaultStrategy / ICallistoVault

REVIEW Main Review

DESCRIPTION

src/external/cdp/interfaces/IDebtToken.sol

- `error CallerNotWhitelisted();`

src/external/cdp/interfaces/IStabilityPool.sol

- `error MaxCollateralsReached();`

src/interfaces/ICDPRouter.sol

- `error CDPRouter_NotUser();`
- `error CDPRouter_NotPermitted();`
- `error CDPRouter_ZeroAmount();`
- `error CDPRouter_NonZeroResidual();`

src/interfaces/ICallistoVault.sol

- `error Vault_MismatchedCoolerAddress();`
- `error Vault_MigratorNotSet();`

src/interfaces/IVaultStrategy.sol

- `error VaultStrategy_ZeroValue();`

RECOMMENDATION

Remove or use unused errors.

RESOLUTION

Status: Resolved.

<https://github.com/CallistoDA0/callisto-core/pull/65>

INFORMATIONAL I-09

RESOLVED

Unused yieldVault Constructor Argument

CATEGORY	Best Practices
LOCATION	CallistoPSM.sol
REVIEW	Main Review

DESCRIPTION

`CallistoPSM` receives a `yieldVault_` constructor parameter and even validates it is non-zero:

```
constructor(Kernel kernel_, address asset_, address collar, address yieldVault_)
Policy(kernel_) {
    require(
        address(kernel_) != address(0) && asset_ != address(0) && collar != address(0) && yieldVault_
        != address(0),
        PSM_InvalidParameter()
    );
    COLLAR = IERC20(collar);
    asset = IERC20(asset_);
    to18DecimalsMultiplier = 10 ** (18 - IERC20Metadata(asset_).decimals());
}
```

However the value is never written to storage or read afterwards. All other constructor arguments are persisted (`asset_`, `collar`), so dropping this one leaves dead logic and reviewers cannot determine whether a yield vault dependency was intended. The current implementation can silently misconfigure the contract because deployments that pass the wrong yield vault address will still succeed and there is no record of the parameter anywhere on-chain. Impact: protocol deployers may believe the strategy address has been wired into `CallistoPSM` when in fact it is ignored, so configuration mistakes go undetected and auditing the live contract provides no hint that the constructor argument was intended to matter.

RECOMMENDATION

Either remove the `yieldVault_` argument entirely (and the accompanying `!= address(0)` check) if it is not needed, or persist it in storage and use it where the contract later references the vault/strategy, so that the constructor input actually matters.

I-09 Unused yieldVault Constructor Argument

CONTINUED

— RESOLUTION

Status: Resolved.

<https://github.com/CallistoDA0/callisto-core/pull/64>

INFORMATIONAL **I-10**

RESOLVED

setCollateralParameters Skips Activation Event

CATEGORY	Warning
LOCATION	AdminContract.sol
REVIEW	Main Review

DESCRIPTION

`AdminContract.setCollateralParameters` force-enables the collateral by writing `collParams.active = true` directly:

```
function setCollateralParameters(...) external override onlyTimelock exists(asset) {
  CollateralParams storage collParams = _collateralParams[asset];
  collParams.active = true;
  _setBorrowingFee(...);
  ...
}
```

By toggling the flag in place, the function bypasses `_setIsActive`, so no `IsActiveChanged(asset, oldActive, active)` event fires, and the `AdminContract__ActiveStatusUnchanged` guard is ignored. If the collateral was already active, this still performs an SSTORE and the monitoring stack will never learn that `setCollateralParameters` activated the asset. Impact: activation events coming from `setCollateralParameters` are invisible to watchers (nothing ever emitted), and redundant writes waste gas when the flag is already true.

RECOMMENDATION

Replace the raw assignment with a call to `_setIsActive(asset, true)` (or at least emit `IsActiveChanged` and return early when `collParams.active` is already true) so activation toggles always follow the same event/validation path and no extra SSTORE occurs.

RESOLUTION

Status: Resolved.

<https://github.com/CallistoDAO/callisto-core/pull/63>

INFORMATIONAL **I-11**

RESOLVED

Unused Admin Slot

CATEGORY	Best Practices
LOCATION	BorrowerOperations.sol
REVIEW	Main Review

DESCRIPTION

`BorrowerOperations` defines `LocalVariablesOpenVessel` with an `AdminContract admin` member, documented as "Reference to the AdminContract for protocol parameters". Inside `openVessel` the struct is instantiated (`LocalVariablesOpenVessel memory v;`) but the `admin` slot is never assigned or read; the function instead uses a separate local `AdminContract admin = AdminContract(adminContract);`. Consequently the struct stores meaningless data and its NatSpec misleads reviewers into thinking it drives parameter reads. The unused slot also forces tools to suppress warnings.

RECOMMENDATION

Remove the `admin` field (and the corresponding NatSpec line) from `LocalVariablesOpenVessel`, or actually assign `v.admin` and use it throughout the function. Eliminating the dead member keeps the struct minimal and prevents unused-field warnings.

RESOLUTION

Status: Resolved.

<https://github.com/CallistoDA0/callisto-core/pull/62>

INFORMATIONAL I-12

RESOLVED

Dead Recovery Branch In WithdrawColl

CATEGORY Best Practices
LOCATION BorrowerOperations.sol
REVIEW Main Review

DESCRIPTION

`BorrowerOperations.withdrawColl` guards the call with `_requireNotInRecoveryMode(v.isRecoveryMode);` before performing any state changes. Even so, the function later wraps the post-withdrawal checks in `if (!v.isRecoveryMode)`. Because the earlier require already guaranteed `v.isRecoveryMode == false`, that branch is dead: the condition is always true, so the compiler still emits a conditional and the code suggests behavior that can never differ. The redundant branch wastes a few gas and increases cognitive load for auditors trying to reason about recovery-mode behavior.

RECOMMENDATION

Drop the `if (!v.isRecoveryMode)` wrapper and execute `_requireICRIsAboveMCR` and `_requireNewTCRIsAboveCCR` unconditionally after the withdrawal logic. This makes it explicit that the checks always run and removes the superfluous conditional.

RESOLUTION

Status: Resolved.

<https://github.com/CallistoDAO/callisto-core/pull/61>

INFORMATIONAL **I-13**

ACKNOWLEDGED

Unused Instruction Struct In Kernel

CATEGORY	Best Practices
LOCATION	Kernel.sol
REVIEW	Main Review

DESCRIPTION

Kernel.sol declares an `Instruction` struct (`struct Instruction { Actions action; address target; }`) alongside the `Actions` enum, but the struct is never referenced anywhere in the kernel, modules or policies. All kernel calls accept the action enum and target address as separate parameters, so the struct only inflates the ABI and misleads readers about an instruction-based interface.

RECOMMENDATION

Remove the unused `Instruction` definition (and any accompanying comments), or refactor the kernel to actually consume `Instruction` objects when executing actions. Keeping only actively used types makes the contract clearer and avoids exporting pointless ABI entries.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-14**

RESOLVED

adminRole Slot Unused In ROLESv1

CATEGORY Best Practices

LOCATION ROLES.v1.sol

REVIEW Main Review

DESCRIPTION

`ROLESv1.RoleData` reserves an `bytes32 adminRole;` word, but the module and its concrete implementation `CallistoRoles` never assign or read that field. Every entry in `_roles[role_]` therefore stores an extra 32-byte slot that serves no purpose, increasing storage costs and implying an admin hierarchy that doesn't exist.

RECOMMENDATION

Remove the unused `adminRole` field (and associated `NatSpec`) from `RoleData`, or add explicit logic that populates and enforces it so the extra storage provides value. Keeping the struct lean avoids wasting deployment gas and reduces the risk of future upgrades misinterpreting the state layout.

RESOLUTION

Status: Resolved.

<https://github.com/CallistoDAO/callisto-core/pull/60>

INFORMATIONAL **I-15**

ACKNOWLEDGED

ensureValidRole Scans Padded Zeros

CATEGORY Best Practices
LOCATION CallistoRoles.sol
REVIEW Main Review

DESCRIPTION

`CallistoRoles.ensureValidRole` iterates all 32 bytes of the role ID and checks each one against the allowed character set. Role identifiers are short ASCII strings padded with `0x00`, so once the loop hits the first zero byte it can safely stop. Instead the function keeps looping through the remaining ~27 bytes, repeatedly re-reading zero and performing the same comparisons. That adds unnecessary gas to every role validation.

RECOMMENDATION

Exit the loop early when `char == 0x00` (after verifying earlier bytes) so padded bytes are skipped altogether. This preserves semantics (zeros are already allowed) while cutting redundant iterations and gas.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-16

RESOLVED

End Price Unreachable At Auction Boundary

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/callisto-core-team1-1759852427511/blob/main/src/external/PSMStrategy.sol#L592
REVIEW	Main Review

DESCRIPTION

The auction uses linear price decay from `startPrice` to `endPrice` based on elapsed time:

```
if (currentTime >= a.endTime) return a.endPrice;  
return a.startPrice - ((a.startPrice - a.endPrice) * (currentTime - a.startTime) / (a.endTime - a.startTime));
```

Separately, `PSMStrategy.closeUnsoldAuctions` closes auctions when it detects they have ended:

```
if (currentTime >= rAuction.endTime) { ... }
```

Because both checks use `>= endTime`, an auction can be closed exactly at `endTime`, the same instant the pricing function first returns `endPrice`. In that boundary case, users have no practical chance to purchase at the final (minimum) price, undermining the intended "last-chance" price point.

RECOMMENDATION

Consider refactoring the condition in `closeUnsoldAuctions`, to be strictly greater than the end time, instead of greater than or equal to. This ensures the auction remains open at `endTime`, allowing purchases at `endPrice` for at least one block before it becomes eligible for closure.

RESOLUTION

Status: Resolved.

<https://github.com/CallistoDAO/callisto-core/pull/75>

INFORMATIONAL I-17

ACKNOWLEDGED

Missing Reserve

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/callisto-core-team1-1759852427511/blob/main/src/policies/CallistoVault.sol#L1033
REVIEW	Main Review

DESCRIPTION

When a withdrawal needs debt tokens and the strategy can't fully cover, the caller fronts the shortfall and receives a withdrawal reimbursement IOU recorded in `withdrawalClaimsWad[account]` and aggregated in `totalWithdrawalClaimsWad`. Users later call `claimWithdrawalReimbursement()` to get repaid in the debt token.

However, `profit()` only deducts interest IOUs (`totalInterestClaimsWad`) from the amount deemed sweepable to the treasury. It ignores `totalWithdrawalClaimsWad`. As a result, `sweepProfit()` can transfer out the very debt tokens needed to repay users' withdrawal IOUs.

Treasury profit is taken ahead of repaying users who covered protocol shortfalls.

RECOMMENDATION

Add `withdrawalClaimsDebt = _convertWadToDebtToken(totalWithdrawalClaimsWad)`; and subtract it alongside interest.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-01**

RESOLVED

Migrate PSM Strategy Could Be DoSed

CATEGORY Validation

LOCATION <https://github.com/GuardianOrg/callisto-core-team1-1759852427511/blob/main/src/external/PSMStrategy.sol#L912>

REVIEW Remediation Review

— DESCRIPTION

H-17 was fixed by introducing an `initializeStateFromMigration` in the PSM strategy that gets called at the very end of the migration that initializes the values of the new strategy according to the old one.

```
// Initialize state of this strategy.  
collarInSP = collarInSP_;  
collarFromPSM = collarFromPSM_;  
collarPendingReplenishment = collarPendingReplenishment_;  
collarDeficit = collarDeficit_;
```

In that function, the code checks if the contract holds any Collar tokens, and if so, they are deposited into the SP (the previous strategy sends the withdrawn Collar directly to the new strategy):

```
// Create an initial COLLAR deposit in the SP.  
uint256 collarBalance = IERC20(COLLAR).balanceOf(address(this));  
if (collarBalance != 0) STABILITY_P00L.deposit(collarBalance, address(this));
```

Later, it checks that the current SP deposits is the same as the one in the previous strategy:

```
// Check that the SP deposit matches the expected value.  
uint256 actualSPDeposit = STABILITY_P00L.calcCompoundedDeposit(address(this));  
require(actualSPDeposit == collarInSP_, PSMStrategy_SPDepositMismatch(actualSPDeposit,  
collarInSP_));
```

However, this is wrong, as it allows anyone to donate any amount of Collar to the new strategy contract and force the above to revert, DoSing the migration.

— RECOMMENDATION

Consider refactoring the require to allow additional deposits:

M-01 Migrate PSM Strategy Could Be DoSed

CONTINUED

```
require(actualSPDeposit >= collarInSP_, PSMStrategy_SPDepositMismatch(actualSPDeposit, collarInSP_));
```

In addition to that, the state updates could also be updated to account for the donated amounts, could reflect what is done `donateCOLLARToReduceDeficit`.

PROOF OF CONCEPT <https://gist.github.com/Ali-Shehab/c59cca33ac058b3611c07a9176d60590>

— RESOLUTION

Status: Resolved.

LOW **L-01**

RESOLVED

Invalid Min Burn Amount Set

CATEGORY	Configuration
LOCATION	https://github.com/GuardianOrg/callisto-core-team1-1759852427511/blob/main/src/policies/CallistoPSM.sol#L76
REVIEW	Remediation Review

DESCRIPTION

Callisto PSM constructor sets the minimum burning amount to 100 Collar, however, it does that by:

```
_setMinBurningAmount(100 * 10 ** (18 - IERC20Metadata(collar).decimals())); // 100 COLLAR
```

The issue is that Collar has 18 decimals, so the calculation results in:

```
100 * 10 * (18 - 18) = 100 * 10 * 0 = 100
```

This means the minimum burning amount is set to 100 wei of Collar, which is almost 0.

RECOMMENDATION

Consider changing the line to:

```
_setMinBurningAmount(100 * 10 ** IERC20Metadata(collar).decimals()); // 100 COLLAR
```

RESOLUTION

Status: Resolved.

<https://github.com/CallistoDA0/callisto-core/pull/111>

§ 05

Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

§ 06

About Guardian.

Guardian is a smart contract security firm. Every engagement is worked by two independent teams competing on the same scope, each with its own tooling and methods, so no single reviewer's blind spot survives.

\$45B+DIGITAL ASSETS UNDER
REVIEW**200+**

ENGAGEMENTS COMPLETED

500+CRITICAL & HIGH FINDINGS
REPORTED

Reach us at audits@guardianaudits.com.